



Department of Applied Informatics and
Information Systems

BSc Thesis

*Towards a domain-specific automated software
development process*

By

George Fakidis

Supervisor: Kostas Vergidis

Thessaloniki, Greece July 2024

Towards a domain-specific automated software development process

Abstract

Software development automation is a promising approach for reducing the cost and complexity of software development. However, existing solutions are inflexible and do not offer a systematic approach that integrates smoothly with the software development lifecycle(SDLC). This thesis proposes a framework for implementing software development automation solutions based on processes, systematic component reuse and structured user input. To achieve this, this approach considers different application domains as different sub-problems to solve in an automated manner that share common tooling. The advantages lie in the proximity to the SDLC and the engineering aspect of software development, while also ensuring vendor lock-in is eliminated and additional cognitive load is kept at a minimum. It is based on domain models reuse, automated decision making for trade-offs regarding architecture and a suite of tools which take care of code generation of simple, repetitive parts such as integrating third-party software and data, and also receive user input with predefined structure based on existing domain knowledge. A tool was developed in order to demonstrate the potential of the approach, specifically in Big Data Systems, demonstrated in a case study based on relevant industry experience.

Table of Contents

[Chapter 1: Introduction](#)

[1.1 Motivation](#)

[1.2 Contribution](#)

[1.3 Contents](#)

[Chapter 2: Key Concepts](#)

[2.1 Model Driven \(Software\) Development](#)

[2.2. Software Development Methodologies\(SDMs\)](#)

[2.2.1 Introduction to SDMs](#)

[2.2.1.1 The importance of SDMs](#)

[2.2.1.2 Heavyweight Methodologies](#)

[2.2.1.3 Heavyweight Conceptual Approach/Assumptions/Axioms](#)

[2.2.1.4 Agile Methodologies](#)

[2.2.1.5 Agile Conceptual Approach/Assumptions/Axioms](#)

[2.2.1.6 Hybrid Methodologies](#)

[2.2.1.7 Hybrid Methodologies Approach/Assumptions/Axioms](#)

[2.2.2 Heavyweight methodologies](#)

[2.2.2.1 Waterfall](#)

[2.2.2.2 Rational Unified Process](#)

[2.2.2.3 Spiral Model](#)

[2.2.3 Agile Methodologies](#)

[2.2.3.1 Scrum](#)

[2.2.3.2 Extreme Programming\(XP\)](#)

[2.2.4 Discussion](#)

[2.3 OpenAPI Specification](#)

[Chapter 3: Related Work](#)

[3.1 Academic Methodologies and Tools](#)

[3.2 Development Automation Tools/Technologies](#)

[3.2.1 OpenAPI Code Generators](#)

[3.2.2 Large Language Models and Generative AI](#)

[3.2.3 Cloud Computing and Cloud Automation Platforms](#)

[3.2.4 Application Containerisation\(Docker\)](#)

[3.2.5 CI/CD](#)

[Chapter 4. The Proposed Approach](#)

[4.1 AS-IS Software Development Process](#)

[4.2 Conceptual Approach/Assumptions](#)

[4.2 Business Context](#)

4.3 Requirements Specification

4.3.1 Functional Requirements

4.3.2 Non-Functional Requirements

4.4 Software Development

4.4.1 Software Types/Taxonomies

4.4.2 Software Architectures

4.4.3 Component Reusability

4.4.3.1 Entity Independent Features

4.4.3.2 Entity Dependent Features

4.4.4 Domain Models Reusability

4.4.4.1 Base Domain Models

4.4.4.2 Domain Model Extensions

4.5 Software Development Process Automation Framework

4.5.1 Analysis and Design Methodology

4.5.2 Implementation of an Automated Process

4.5.3 Modelling(BPMN) Software Development Methodologies

4.5.3.1 SCRUM

4.5.3.2 Waterfall

4.5.3.3 Spiral Methodology

4.6 TO-BE Software Development Process

Chapter 5: Concrete Methodology for Data Services

5.1 Data Sources Automation

5.2 Dashboards Automation

5.3 Analytics Module Automation

5.4 Export/Reports Module

5.5 Data Storage Module

5.6 Functional Specification

5.7 Non-Functional Specification

5.8 Business Context Required

Chapter 6: InfoDev, Automation Component for Data Services

Chapter 7: Case Study on Energy Data for Energy Related Forecasting

7.1 Scenario Outline

7.2 InfoDev Demonstration

Limitations

Further Research

Conclusion

References

List of Figures:

[Figure 4.1: Proposed Framework](#)

[Figure 4.2: Traditional Software Development process flow\(BPMN\)](#)

[Figure 4.3.a: SCRUM development process-high level overview\(BPMN\)](#)

[Figure 4.3.b: SCRUM development process-pregame\(BPMN\)](#)

[Figure 4.3.c: SCRUM development process-sprint\(BPMN\)](#)

[Figure 4.3.d: SCRUM development process-postgame/closure\(BPMN\)](#)

[Figure 4.4.a: Waterfall model-Core process \(BPMN\)](#)

[Figure 4.4.b: Waterfall model-Preliminary Program Design \(BPMN\)](#)

[Figure 4.5 Spiral Methodology \(BPMN\)](#)

[Figure 4.6: Modernised Version of Software Development process flow\(BPMN\)](#)

[Figure 5.1: Big Data Systems high level overview](#)

[Figure 5.2: UI layer in Big Data Systems](#)

[Figure 6.1: Home Page of InfoDev](#)

[Figure 6.2: User names the software to be created in InfoDev](#)

[Figure 6.3: User submission of data source\(OpenAPI Specification\)](#)

[Figure 6.4: Data selection from third party system](#)

[Figure 6.5: Metadata for data sources page](#)

[Figure 6.6: Layout creation page](#)

[Figure 6.7: Non Functional Requirements page](#)

[Figure 6.8: Modal for user action based on the submitted input](#)

[Figure 6.9: Code generation page in selected language](#)

[Figure 7.1: Step 1: Select software type](#)

[Figure 7.2: Step 2: Submit software name](#)

[Figure 7.3: Step 3: Generate name using AI and submit it](#)

[Figure 7.4: Step 4: Submit an OpenAPI Specification to import the data](#)

[Figure 7.5: Step 5: See specific data types from data source](#)

[Figure 7.6: Step 6: Select data types](#)

[Figure 7.7: Step 7: Submit metadata for each data type](#)

[Figure 7.8: Step 8.1: Create layout for energy generation data](#)

[Figure 7.9: Step 8.2: Create layout for energy demand data](#)

[Figure 7.10: Step 9: Submit Non-Functional Requirements for Big Data System](#)

[Figure 7.11: Step 10: Choose code generation or requirements document print.](#)

[Figure 7.12: Step 11: Generated Requirements Document](#)

[Figure 7.13: Step 12: Choose programming language for codegen](#)

[Figure 7.14: Github Repository created](#)

[Figure 7.15: Github Issues generated from user input](#)

[Figure 7.16: Details of Github Issue #1](#)

[Figure 7.17: Details of Github Issue #2](#)

[Figure 7.18: Frontend code generated in github repository based on user input](#)

[Figure 7.19: Backend code generated in github repository based on user input](#)

Chapter 1: Introduction

1.1 Motivation

Software development is considerably costly and time-consuming as can be seen in many software development related statistics such as [1], [2]. More specifically, time spent in already understood software domains, technical matters and inadequately integrated processes is time that can be saved. Automation efforts are limited in their scope and mostly revolve around automating narrow and specific problems within the SDLC and not the wide problem.

1.2 Contribution

In this work multiple contributions towards an automated software development process, of varying importance, can be identified. Traditional Software Development Methodologies(SDMs) were modelled using modern modelling tools(BPMN), the business context was associated with software related decisions and methodologies were combined in order to automate software development in a domain-specific manner. All of this effort was the foundation for a tool that facilitates Big Data Systems development.

1.3 Contents

In chapter 2 we give an overview of the key ideas and concepts that will be required to understand the proposed approach. In Chapter 3 we investigate the related work on the topic, in both academic and industrial contexts. In Chapter 4 we present in detail the suggested approach for the implementation of software development automation solutions. In Chapter 5, we focus on how the domain of Big Data Systems can be automated and in chapter 6 we demonstrate a tool that is based on the aforementioned analysis. Finally in Chapter 7, we showcase a case study in order to see how the tool is used in a practical example, which is followed by a brief discussion, in Chapter 8, of the existing limitations and the future directions.

Chapter 2: Key Concepts

2.1 Model Driven (Software) Development

Model-driven development (MDD) prioritises creating and utilising models throughout the software development process. These models act as blueprints, capturing the system's functionality, data, and behaviour. MDD emphasises iterative refinement of these models by different specialists throughout the development lifecycle. This ensures a clear understanding of the system's requirements and facilitates communication between different teams. By leveraging these models, MDD aims to automate tasks, improve code quality, and reduce development time.

Its principles can assist the automation of development processes by providing efficient solutions to narrow and specific problems as MDD also supports code generation methods which can be useful throughout the software development process.

A more concrete approach that captures the essence of MDD is MDA, model driven architecture(MDA), an approach supported by the Object Management Group(OMG). MDA separates business and application logic from underlying platform technology thus providing flexibility in the implementation details and by reducing the problem of coupling implementations to a specific programming language and its related ecosystem. Its techniques are encapsulated in the following models that can be iteratively developed and reused in different contexts.

The first view of a business scenario is its domain model. The domain model, in MDD terminology, is discussed as the Computation Independent Model(CIM), which describes the system without any computation related details, providing an ontology of the problem domain.

Platform-independent models(PIMs) of an application or integrated system's business functionality and behaviour, provide a detailed representation of the business functionality and behaviour of an application separate from the technology-specific code that implements it. Those models can then be converted into a Platform-Specific Model(PSM) and then actually deployed in a platform such as C# .NET and Java EJB.

2.2. Software Development Methodologies(SDMs)

2.2.1 Introduction to SDMs

Developing software is about starting from an idea(or requirements depending on the case) and iterating upon it with feedback until the deliverable is satisfactory [3].

Towards a domain-specific automated software development process

Software differs from traditional construction projects in one fundamental way: Design and Implementation are not separate sequential phases but steps meant to be iterated upon. In construction the irreversibility of decisions made earlier dictate the separation of Design and Implementation and the precision with which designs can be executed according to specification.

Software Development methodologies provide the concrete details in the general image of the software development life cycle(SDLC). The SDLC consists of the following four fundamental phases: Planning, Analysis, Design, Implementation [3].

Explained shortly, **Planning** relates more to the current business context and is concerned with the business needs and the allocation of resources towards the development of the software [3]. **Analysis** amasses the information required to build a system, focused on requirements gathering and a more formal system request where the decision to develop the software or not is made. **Design** is the part where technical details start being discussed. A high level architecture of the system, the user interface, how data will be stored and what sub-programs should be written. Also in this stage it is discussed whether the development will happen in-house or outsourced or done by adapting existing software to the company's needs. **Implementation**, the last phase, includes the actual development of the system, its deployment(which can be done in different ways depending on the business case) and lastly a support/maintenance plan.

Every phase can be analysed further into more detailed steps, where guidelines are provided and which produce deliverables(we will be referring to deliverables as artefacts from now on, as it is an important concept later on). It is important to know that in every case the SDLC might differ [3], but the general concept remains the same: How should Software Development work be analysed in order to manage its complexity.

So Software Development methodologies are the frameworks that dictate the sequence of those phases, the process flow, how the phases are broken down into individual steps and how individuals should coordinate their work.

2.2.1.1 The importance of SDMs

SDMs are the procedures that handle the discovery of the **domain** problem, the client **requirements** and the appropriate and **feasible technical solutions** within the business context(budget,timeline,experience of employees etc etc.).

Choosing the appropriate methodology depends on specific project characteristics such as objective, scope, requirements, resources, architecture and size will determine which methodology suits them best [4].

2.2.1.2 Heavyweight Methodologies

Heavyweight software development methodologies, often referred to as traditional approaches, prioritise detailed planning, extensive documentation, and a structured design process. These methodologies follow a sequential order, like the Waterfall model, where each phase (requirements gathering, development, testing, deployment) is completed before moving to the next. This emphasis on upfront planning aims for a clear roadmap and minimises surprises during development. However, heavyweight methodologies can be inflexible in adapting to changing requirements or new technologies.

2.2.1.3 Heavyweight Conceptual Approach/Assumptions/Axioms

The effectiveness of such methodologies is based on implicit assumptions that should be studied in order to understand their suitability for different scenarios. A non-exhaustive list of critical assumptions follows:

- No fundamental changes will happen in the requirements.
- Planning upfront is useful (which is evidently not always the case).
- Requirements are known, or can be discovered through systematic study without actual functional prototypes.
- Communication needs to be formal, as in structured detailed documents.

2.2.1.4 Agile Methodologies

Agile methodologies, in contrast to heavyweight approaches, value flexibility and rapid iteration. They break down projects into smaller, manageable chunks called sprints, allowing for continuous development and adaptation. Agile teams prioritise collaboration and communication, with frequent feedback loops to ensure the final product aligns with evolving needs. This adaptability makes them ideal for projects with uncertain requirements or where new technologies might emerge during development. However, the lack of a rigid structure in some agile methodologies can lead to challenges with project scope creep or unclear deadlines.

2.2.1.5 Agile Conceptual Approach/Assumptions/Axioms

Agile methodologies approach development from the opposite perspective of heavyweight methodologies. This means that the assumptions of agile methodologies

Towards a domain-specific automated software development process

can be best explained by inverting those of heavyweight methodologies. This can also be easily noticed by viewing the [Agile Manifesto](#)'s core principles as listed below:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- over comprehensive documentation
- Customer collaboration over contract negotiation over contract negotiation
- Responding to change over following a plan

2.2.1.6 Hybrid Methodologies

Hybrid methodologies bridge the gap between heavyweight and agile approaches. They combine elements of both, aiming to leverage the strengths of each. Here, projects benefit from the upfront planning and clear structure of traditional methods, often in the initial phase, while incorporating the iterative development and adaptability of agile throughout. This allows for a solid foundation while enabling adjustments based on user feedback or changing requirements. One of the most popular instances of such a hybrid approach is the spiral model, which has an iterative agile nature, combined with the planning tendency of heavyweight methodologies.

2.2.1.7 Hybrid Methodologies Approach/Assumptions/Axioms

Hybrid methodologies are a more nuanced approach towards software development trying to combine elements of both ends of the spectrum and allow a certain degree of adaptability to the situation at hand. The assumptions are minimal and can be generally summarised as the statement that “While X is important, it should not be included in unfavourable situations”.

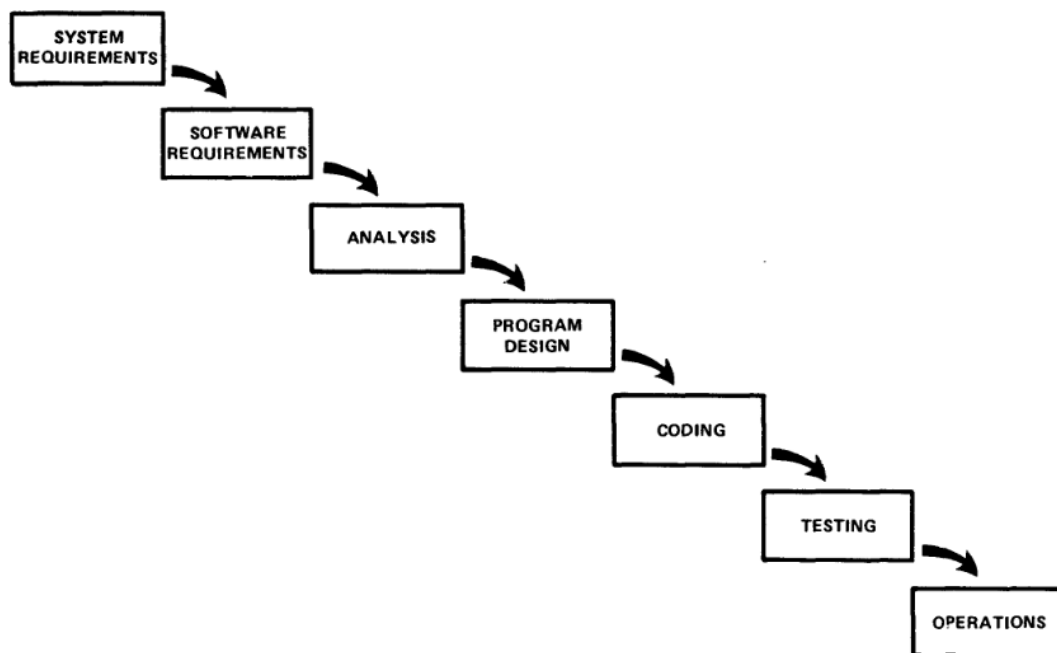
2.2.2 Heavyweight methodologies

2.2.2.1 Waterfall

Introduced in the 70s, waterfall set the baseline of software development methodologies. Winston W. Royce, the first published author of the waterfall model (a name that came later from a different author) acknowledges the necessity of a methodology for developing large software systems while noting that for systems smaller than a certain size the only true development steps required are analysis and coding [5].

Before starting our discussion it is also important to note that the historical context was centred around mostly scientific computations targeted for usage in very specific scenarios with very constrained technological resources, unlike today's complex landscape where apps have millions of users and a myriad of different devices. As

evident many concepts delved into the original work seem unusual for today's data and the average project of our time e.g. "Allocate Subroutine storage", "Allocate subroutine execution times". In the original paper [5], the author iterates upon the initial design as pictured below and, adds a preliminary program design step("Step 1: Program Design comes first"), strict documentation requirements throughout the steps("Step 2: Document the design"), an extra implementation step for critical parts ("Step 3: Do it twice") and finally testing procedures("Step 4: Plan, control and monitor testing").



In the waterfall model as explained in [5] **documentation is of the utmost importance** as it is an artefact that supports all development and management efforts [5]. Phases generally are thought to generate large amounts of documents that serve as the communication between different collaborators. This is one of the reasons why this methodology is not suitable for smaller projects and is considered a heavyweight methodology, due to the upfront planning and design that is required to be done.

2.2.2.2 Rational Unified Process

The Rational Unified Process (RUP), introduced in [6], provides a structured framework for software development. It emphasises collaboration and iterative

Towards a domain-specific automated software development process

refinement to ensure a clear understanding of the system's purpose and domain knowledge.

The core principles/concepts it operates on are:

- **Roles, Activities, and Artefacts:** RUP defines different roles (workers) with specific responsibilities. These workers perform activities within defined workflows (e.g., design, implementation, testing) to produce tangible deliverables (artefacts).
- **Iterative Refinement:** Throughout the development process, various models (e.g., business use cases, system architecture) are created and progressively refined by different roles. This ensures a comprehensive understanding as the project progresses. These models serve as a shared language and blueprint, guiding development decisions and facilitating collaboration across different teams.

RUP outlines core workflows specific to software development activities, along with additional workflows for project management and configuration management. Documentation also follows a similar workflow pattern, highlighting its importance as an integrated part of the development process.

The process itself is divided into distinct phases:

- **Inception:** Here, a promising idea is transformed into a clear vision for the final product, including a justification for its development (business case).
- **Elaboration:** This phase focuses on detailing the system's functionalities (use cases) and designing its overall architecture.
- **Construction:** During this phase, the actual software is built, fleshing out the previously designed architecture.
- **Transition:** The final phase prepares the system for release. A limited group of experienced users (beta testers) try the product and provide feedback on any issues.

It should be noted that for each workflow discussed, different subtasks happen during different phases and each workflow has a different level of activity throughout the different phases. For example, in the transition phase, minimal software development related work is done and more attention is paid to the relevant user documentation.

2.2.2.3 Spiral Model

The spiral methodology is a software development process that prioritises risk management throughout the entire project lifecycle [7]. It combines elements from traditional waterfall and prototyping methodologies to create a flexible and iterative approach. Here's a deeper dive into its key characteristics:

- **Risk-Driven Iterations:** Unlike the linear progression of waterfall, the spiral model adopts a cyclical approach. The project progresses through phases like planning, design, implementation, and evaluation, but each phase can be revisited and adapted based on the risks identified at that stage. This allows for early detection and mitigation of potential problems, preventing them from snowballing later in development.
- **Focus on the Biggest Risks First:** Within each iteration, the team prioritises the most significant risks for that specific stage and attempts to resolve them. This ensures that resources are allocated efficiently, and the development effort tackles the most critical challenges first.
- **Integration of Prototyping:** The spiral model incorporates elements of prototyping. Early prototypes can be developed within each iteration to validate design decisions and uncover potential risks before significant development resources are committed.
- **Flexibility and Adaptability:** The cyclical nature of the spiral model allows for course correction throughout the project. New information or changing requirements can be readily addressed by revisiting previous phases and adapting the development plan as needed. This is particularly beneficial for projects with high levels of uncertainty or where requirements might evolve during development.
- **Clear Decision Points:** The spiral model defines clear decision points at the end of each iteration. Here, the project team evaluates the progress made, the risks encountered, and the resources expended. Based on this assessment, they can decide to continue to the next iteration, revise the plan, or even terminate the project if necessary.

2.2.3 Agile Methodologies

2.2.3.1 Scrum

Scrum, as originally presented in [8], differs significantly from the modern version of it that most people are familiar with. They share the conceptual underpinnings but the practical application is not that similar and the traditional approach has less defined practices unlike today (planning poker, daily standups etc. etc). Software is not delivered all at once but is separated on functional deliverables that create some value independently of other components. Scrum relies on the assumption that software development itself cannot be controlled, but we should strive to control the surrounding process and environment, and provides guidelines for every aspect of the process(project teams, estimation, deliverables etc. etc.).

Towards a domain-specific automated software development process

The controls in the methodology are the backlog, releases, packets, changes, problems, risks, solutions and issues that can be studied in more detail in [scrum paper]. Backlog and releases are the primary components of the scrum methodology. Backlog consists of desired features that are not yet implemented in the software and bugs that have not been fixed yet. Releases are defined as a set of backlog items that make sense as a software release from a business standpoint.

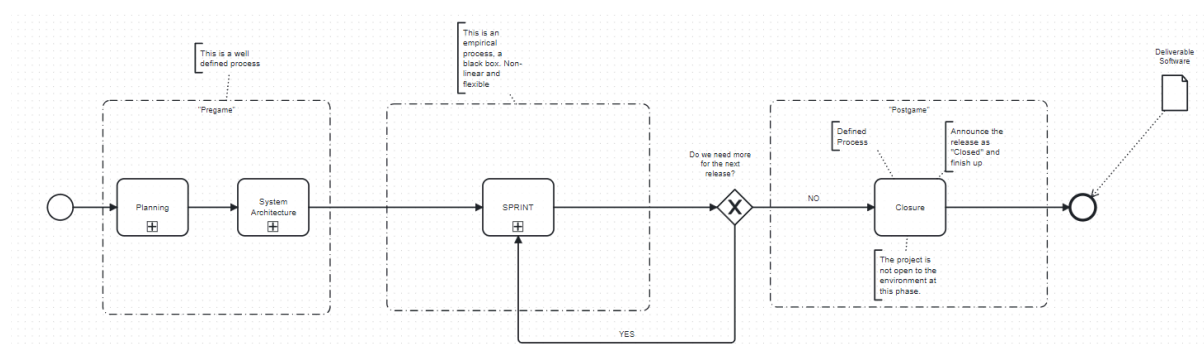
The scrum process is broken down into 3 phases:

1. Pregame
2. Game
3. Postgame

The pregame phase is mostly concerned with **planning** and **system architecture**. In planning, the primary aim is to define a new software release by considering the existing desired features of the customer (backlog) and the schedule and cost variables. In architecture, a high-level design is made for how the selected backlog items will be implemented.

In the game phase, software development occurs in accordance with the planning that has been done, releasing the new functionality that has been agreed upon, separated in **development sprints**.

In the postgame phase, the primary tasks related to **closure** occur, like preparing the related user manuals and marketing materials, generally tasks required to ensure that the release is complete in a business sense. At a high level, scrum can be understood by the following diagram.



2.2.3.2 Extreme Programming (XP)

Unlike traditional methods with their lengthy planning stages, XP by Kent Beck thrives on short, iterative development cycles [9]. XP takes the following statement to its logical extreme: “If long development cycles were bad because they couldn’t adapt to changes, perhaps what we needed was to make shorter development cycles”. It recognises that its application should be preferred for small to medium sized systems where requirements are not known upfront and will most likely change. This assumption should be considered when evaluating the effectiveness of the various strategies suggested. It will also be evident below that outside of that case it is greatly ineffective as many of the practices recommended cannot be applied.

Its presentation avoids theoretical processes and is more focused towards programming practices and development cycles. Kent Beck also provides information for how to handle common problems, such as turnover and changing requirements. Also very specific practices that enhance the process are prescribed and should be briefly discussed.

A high-level overview of the practices follows:

- **Planning game:** Customers start off the process by deciding the scope and timing of releases based on effort estimates provided by software developers. Only those specific stories are actually implemented, nothing extra.
- **Small releases:** The system is put into production early in the project, surely before completion of the project. Releases should be frequent.
- **Metaphor:** The shape of the system is communicated with easily understood analogies to ensure understanding from all stakeholders. This bridges the communication gap between technical and non-technical people.
- **Simple design:** Simplicity is key, code should not be duplicated and not an overly complex system should be made if it is not absolutely necessary.
- **Tests:** Many unit tests should be written (Test-Driven-Development). Customers write functional tests for the stories in an iteration. In normal scenarios all tests should pass before releases (for business reasons this rule can be broken if deemed necessary)
- **Refactoring:** The codebase is properly maintained with its structure simplified while also passing existing tests.
- **Pair programming:** Code is written by two people at each workstation.
- **Continuous integration:** When integrating, the system is built from scratch and all tests must pass or the changes are discarded.
- **Collective ownership:** Every programmer improves any code anywhere in the system at any time if they see the opportunity.
- **On-site customer:** The customer works along with the team full-time.

Towards a domain-specific automated software development process

- **Just rules:** The rules should be followed, but not blindly, they can be changed through team discussion. However, the team should agree on how to assess if a new rule is effective, for the team to pragmatically evolve its rules.

The team strives for simplicity in the codebase, aiming for clean and maintainable solutions. Frequent feedback loops ensure that the project stays on track and that any issues are identified and addressed quickly. XP also encourages courage, empowering developers to make decisions and experiment with new approaches.

This combination of iterative development cycles, core values, and specific practices like pair programming (where two programmers work together on one station) sets XP apart.

2.2.4 Discussion

It is now evident that there is not an globally optimal method, for each software development project there are constraints and unique characteristics that guide the selection of a process/methodology. Methods themselves recognise their suitability for one type of projects and their weaknesses for another type of software projects. So it is important to develop the characteristics of the methodologies and of the software types to enable a reduction of the overall software development effort.

The main things that separates agile from heavyweight methodologies are the underlying assumptions about creating software. Heavyweight methodologies assume that complexity and change can be tamed by rigorous processes and extensive planning, while Agile methodologies believe that to handle complexity and change you need to be as flexible as possible and create software as soon as possible to begin the external feedback loop process and deliver value incrementally. Heavyweight methodologies rely more on official documents being reviewed throughout the project with the implementation being just another step in the whole process. Agile methodologies are working/usable software oriented, so implementation throughout small development cycles(which implicitly include analysis and design but without the burden of producing formal documents, but working code) is the primary driver of progress in the project. So methodologies align themselves in a spectrum based on how much planning-analysis-design is done beforehand and the rigour/strictness the processes impose on the software development environment.

2.3 OpenAPI Specification

The OpenAPI Specification (OAS) acts as a standardised language for describing RESTful APIs. It defines the structure and functionalities of an API in a clear and machine-readable format, typically written in YAML or JSON.

Its contents can be summarised in the following manner:

- **Core Information:** This section identifies the OAS version and provides details about the API itself, such as its title, description, and contact information.
- **Paths:** This section defines the different endpoints (URLs) available within the API. It details the supported HTTP methods (GET, POST, PUT, etc.) for each endpoint and what data should be included in a request (in either body, headers or query parameters) and what are the possible responses.
- **Components :** This section acts as a central repository for reusable definitions. It can house data models (schemas) describing the format of request and response data, common headers used throughout the API, and even security information.

This format facilitates collaboration between developers and API consumers by providing a shared understanding of the API's capabilities.

Chapter 3: Related Work

3.1 Academic Methodologies and Tools

In [10] the authors have created a framework to automate the selection of an architecture for a software system based on user input. More specifically the framework receives as input the application domain, the application type, the non-functional requirements, the application features and the application features and outputs for each of the available architectures an evaluation score(a percentage value). The framework is based on a literature review in [11], where they conducted interviews with domain experts.

In [12] the author is reporting results on a large-scale empirical case study whose purpose is to separate in categories the requirements in the web-based Enterprise Systems domain. In that work several observations are useful to the pursuit of automated software development processes. The most important one is that requirements in the studied systems tended to be concentrated in a few categories rather than scattered throughout the requirements space[12]. In a similar way, the functional requirements for the studied domain are composed primarily of a few core requirements types [12]. It is also noted that the classification of functional requirements is a less researched area, and that existing efforts are generally too abstract and not detailed and comprehensive enough.

In [13], [14], [15] the authors present different MCDM models integrated in a DSS system that can be found in [this website](#). The tool supports an automated selection process for a plethora of common software related tools/technologies. Generally the decision makers select the desired features and assign their importance to them in a MoSCoW way(Must Have,Should Have,Could Have, Won't Have). As output there is a set of feasible solutions that the user can choose from. The methodology revolves around Qualities and Features and to what degree they are satisfied by different Alternatives. The Features rely on evaluations from domain experts. The Qualities are from the ISO/IEC 25010 and the Ext. ISO/IEC 9126. The Alternatives are mostly derived from existing literature, documentation,reputable aggregation websites etc. etc. This approach assists professionals by significantly reducing the time to

In [16] the authors present techniques to translate a specific subset of BPMN models into BPEL executable specifications, implemented in the [BPMN2BPEL library](#). This is important as it provides a user-friendly, semantically rich and expressive way(BPMN

19

models) for a user to define the logic for his application, with the developers only providing the building blocks (services invoked by BPEL) and not limiting the software's functionality.

In [17] the authors adopt classical Artificial Intelligence methodologies, more specifically from the domain of Automated Planning, in order to solve the problem of Architecture Evolution, meaning transitioning from an initial architecture to a target architecture. Being able to automatically evolve the architecture of the software is important and is part of the maintenance stage of the software development life cycle (SDLC).

In [18], [19], [20] the authors conceptualised and implemented various aspects of automated testing based on the OpenAPI Specification (OAS). More specifically, in [18] the authors introduce a DSL that assists in automatic testing of a REST API with an approach that explicitly considers dependencies between API calls and constraints. In [20], the authors present a methodology and its implementation to facilitate property-based testing by automatically generating tests that check common properties of REST APIs. In [19], a similar model-based approach is used to generate test cases for REST APIs in an automated way by taking advantage of the OpenAPI Specification. Testing is a core component for achieving durable, maintainable systems and is widely recognized as part of the SDLC.

In [21], [22], [23], [24] work is done to automate different parts of REST APIs by providing extensions and using the OAS. In [21], an OAS extension is proposed in order to automatically generate REST APIs for Command-Line Applications without modifying the original CLI program. In [22], the RestMule framework for handling service policies, e.g. rate-limits, pagination etc. etc. is showcased. This tool combined with an OpenAPI Specification can automate the generation of reliable clients that respect service policies and thus fetch data in a simple and consistent manner. In [23], an extension of the OAS is demonstrated, IDL4OAS, that captures inter-parameter dependencies (a value of a parameter determines the valid values of other parameters), a phenomenon in APIs that is only informally mentioned in API documentations. This powers the generation of code that makes those dependencies explicit and throws an error when they are not satisfied. All of this work demonstrates the potential for further automation and more robust systems, giving tools to reduce development man-hours.

In [25], Modelbus, an integration framework for tooling supporting the software development process, that addresses common concerns within the automation of development processes has been developed. Its purpose is to enable a generic standards-based cooperation between applications with no explicit interconnections. It makes use of well established standards (such as BPMN, BPEL and SPEM) and utilises the SOA principles.

3.2 Development Automation Tools/Technologies

3.2.1 OpenAPI Code Generators

OAS unlocks the potential for automation through a rich ecosystem of tools that can be explored [here](#). A notable mention is the [openapi-generators](#) tooling that can generate client libraries and server code based on the OAS definition, in a plethora of programming languages, assisting in the development effort. Additionally, they can automate API documentation creation, testing procedures, and even generate mock servers for development and testing purposes. This synergy between the OpenAPI Specification and automation tools has plenty of potential to shorten the development timeline.

3.2.2 Large Language Models and Generative AI

Large language models (LLMs) are emerging as capable tools to streamline software development processes and drive down costs and time-to-market. These AI models, trained on massive datasets of code and natural language, offer several capabilities that directly reduce development time. LLMs can generate code snippets or even entire functions based on natural language descriptions, automating repetitive coding tasks. Additionally, they can analyse existing code and suggest improvements, helping developers identify potential bugs or inefficiencies. Furthermore, LLMs can translate technical documentation into simpler language or generate documentation from code itself, improving communication and knowledge transfer within development teams. Overall, by automating tasks, improving code quality, and enhancing communication, LLMs hold significant potential to expedite the software development process. There are no best practices surrounding LLMs and writing code yet, and the services built atop of LLMs have not yet been widely adopted but there is great potential for automation of SDLC tasks. We will discuss two specific tools that stand out among the rest and can greatly reduce development efforts.

[GitHub Copilot](#), based on [OpenAI's Codex](#) model trained on Github's Data, is an AI tool that acts as a coding assistant for programmers. It analyses the code you're working

on and suggests completions, similar to autocomplete but for entire lines or code blocks. It can also understand natural language comments, so you can describe what you want the code to do and receive suggestions. This can save developers time and effort by reducing the amount of code they need to write themselves.

[Figma](#), a web-based design tool for creating user interfaces, allowing teams to collaborate on designs in real-time, recently integrated AI functionalities aiming to streamline the design process. These features encompass enhanced asset search, text manipulation (translation, shortening, rewriting), content generation for mockups, one-click prototyping, automatic layer naming, and even design generation based on text prompts. This allows designers to focus on core design activities such as user experience refinement, concept iteration, and overall design strategy [26], meaning shorter and more effective design processes.

3.2.3 Cloud Computing and Cloud Automation Platforms

Cloud computing has revolutionised software development by providing on-demand access to computing resources and abstracting away the difficulties of maintaining physical servers on-premises. In its beginning Cloud Computing used to mostly offer primitives revolving around server instances, configuring networks and other low-level functionalities. As time has passed Cloud Computing platforms have started to offer services that are closer to the development cycle. These services leverage cloud infrastructure to automate various aspects of software deployments. They can handle tasks like scaling, infrastructure management, domain name registration and usage, automatic SSL integration and security(e.g. DDOS protection) freeing developers from manual configuration and server maintenance. Some of the most popular platforms that offer services that automate a whole class of tasks are [Vercel](#), [Nuxthub](#) and [Forge](#), that integrate to the level of programming framework used to provide a frictionless experience. Utilising those can considerably drive down time-to-market, although long term cost-wise it might not be the cheapest alternative, especially for large enterprises with applications that are used by millions of users.

3.2.4 Application Containerisation(Docker)

Containerization is a virtualization technique that packages an application with all its dependencies into a standalone unit called a container. These containers share the host operating system's kernel but run in isolation from each other. This ensures consistency and portability, since the application runs the same way regardless of the environment. Docker is the leading containerization platform, although many similar

technologies exist such as the open-source alternative, Podman. Tooling is provided by different vendors to build, deploy, and manage these containers, making it easier for developers to build and run applications consistently across different environments. This means that there is no need for manual deployment processes as by writing the container instructions it can be deployed with ease in all popular cloud providers such as Amazon Web Services(Azure),Google Cloud and Microsoft Azure.

3.2.5 CI/CD

CI/CD, short for Continuous Integration and Continuous Delivery/Deployment, automates the software development pipeline. With CI, when developers merge their code changes into a central repository(Github/Gitlab), automated testing is automatically triggered, identifying bugs early on. CD takes it a step further by automating the deployment process itself by making use of containerisation and other technologies. This means new features and bug fixes can be delivered faster and with higher confidence as failed tests block the deployment. By automating these tasks, CI/CD frees developers from repetitive work, allowing them to focus on writing new code and functionalities. This significantly reduces development time and gets software into production quicker.

Chapter 4. The Proposed Approach

This proposed framework separates work into 4 packages in order to clearly define boundaries and enable cohesion within the packages.

The first package is the **business context**. It includes information for the business that is considered (most times implicitly and should be made explicitly in a formal theory) when making decisions regarding the software project. It will be explained in detail in the next paragraphs.

The second one is **process automation**, and it includes a discussion of how to integrate automation into a businesses' operations to better handle their software development projects.

The third package is **requirements gathering**, as it is of the utmost importance to optimise the requirements gathering process in order to improve the software development methodology in general. It is important to note that apart from non-functional and functional requirements.

The fourth package is **software development**, where the more technical aspects of software engineering are discussed, such as architecture, software type taxonomies, code generation, all of the ingredients needed in order to make the automation efficient and effective.

Automating the software development process in a generic way is not fruitful as covering the edge cases in a complete manner will result in a contradicting abstraction that should both provide high-level mechanisms to actually consider it automation, but access to lower level details in order to solve minor issues that arise.

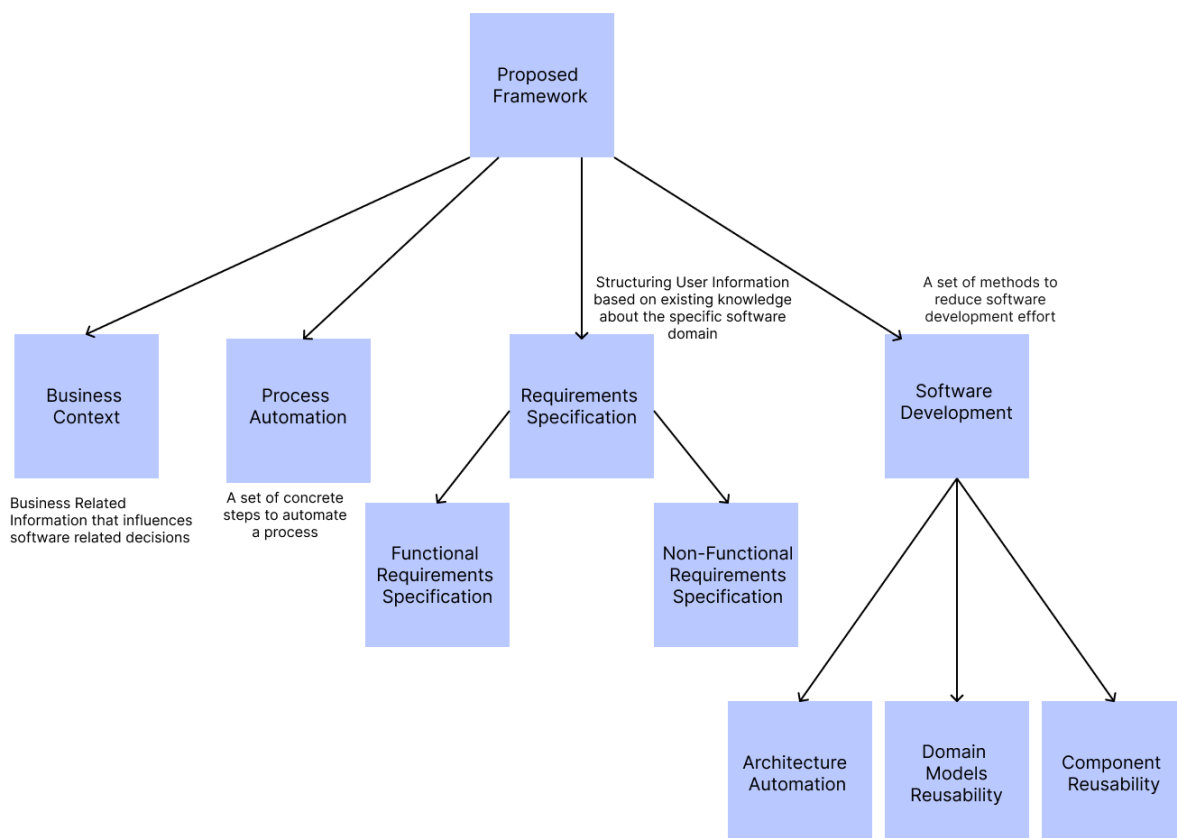


Figure 4.1: Proposed Framework

4.1 AS-IS Software Development Process

To understand the value of an automated approach, we should seek to understand the existing approach. A simplified model of a generic software development process can be seen below. It abstracts away the iterative nature of software development as this mostly revolves around repeating certain steps with more information regarding the problem domain and the customer's requirements. The main roles are that of the Customer and the Company, which contains the sub-roles of Analyst, Technical Lead (Senior Developer) and the whole Development Team. The important thing to note here is that all tasks are not automated and involve considerable human resources costs.

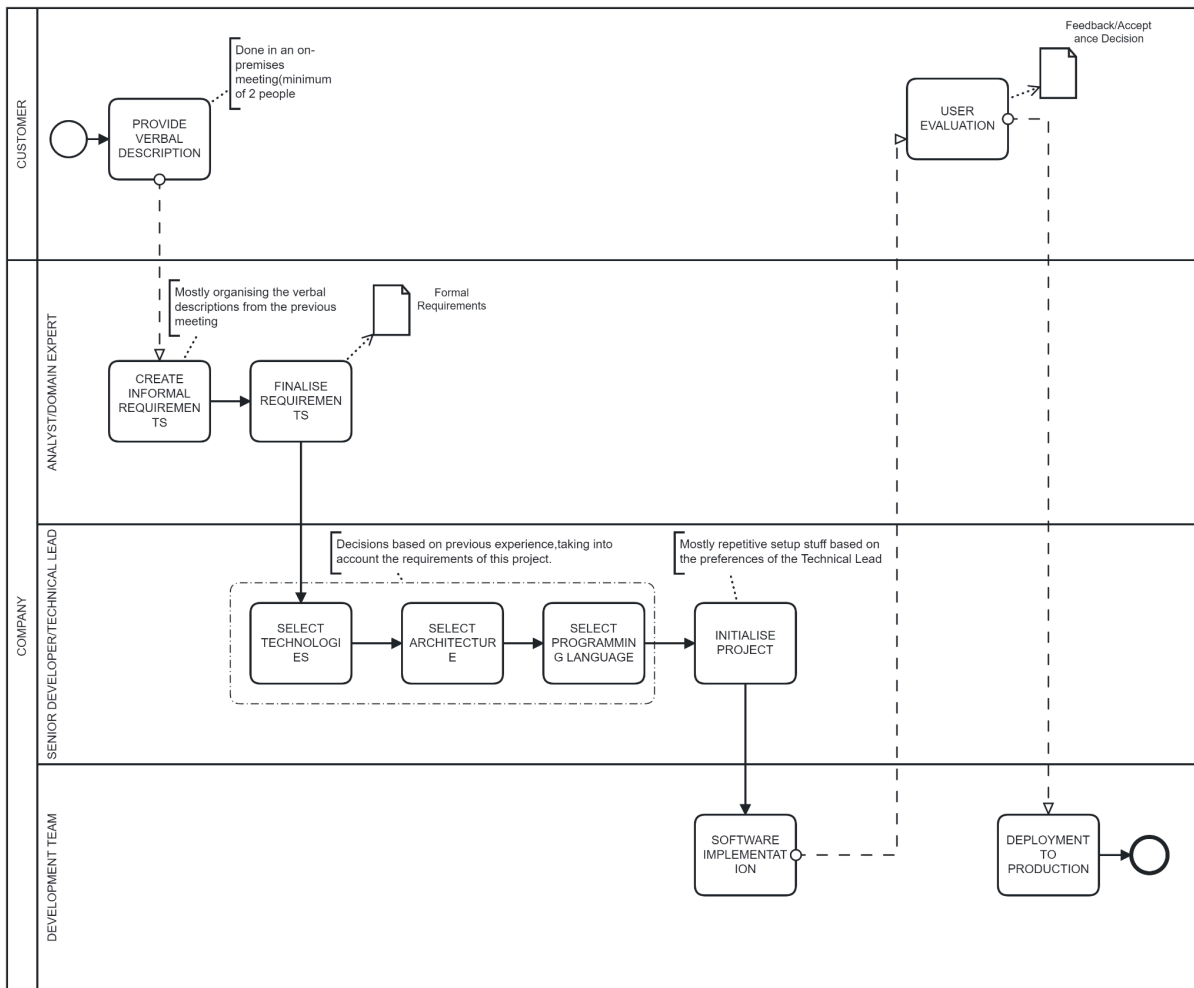


Figure 4.2: Traditional Software Development process flow(BPMN)

Software development processes are the tool we use in order to tackle the complexity of taking an idea and turning it into executable software. A simplified one-dimensional version of the whole process could be seen in the following steps:

1. Verbal Description
2. Informal Requirements
3. Formal Requirements
4. Technology Selection(Database Provider,Cloud Provider etc. etc.)
5. Architecture Selection
6. Programming Language Selection
7. Project Initialisation
8. Implementation
9. Acceptance Test
10. Installation/Deployment

Automating the software development process should be about compressing each step, enabling efficient communication channels between the people involved and

Towards a domain-specific automated software development process

ensuring that standards are met throughout the effort. Going from idea to a 100% executable deliverable software in one go is not feasible and not even desirable because the whole process is iterative and feedback can be given at most points in time. The malleability of software is a property that should be taken advantage of and not eliminated for the sake of automation, automation is not a goal itself but a way to drive down costs and enhance quality.

The first step “Provide Verbal Description” involves the customer and some kind of representative from the company that will be developed. On premises meetings include travel costs, organisational costs and the man-hours required to actually interact with someone to understand his situation and receive the information. The representative has a mental model for what he needs to receive as information and will ask the right questions in order to obtain those. Also it is important to understand the broader context of the customer and have an accurate image of his business case/situation.

We will now discuss the steps “Technology Selection”, “Architecture Selection”, “Programming Language Selection” in conjunction as they are the technical steps that involve solely technical personnel.

- **Technology selection** more specifically is concerned with the various technologies involved in a common software project, such as cloud providers and database systems.
- **Architecture selection** is about considering the trade-offs between different architectural styles such as Microservices, Service Oriented Architecture and Layered Architecture. Those decisions need careful consideration of the specific case and require a certain amount of knowledge and experience to be made in an effective way and also require some time to get right even for seasoned experts.
- **Programming language selection** is about choosing the programming language, which automatically also selects the programming paradigm. In most programming languages there is a first-class paradigm that is supported fluently in the language, and then elements from different paradigms are also integrated into the language(e.g. The Streams API in Java is inspired by functional programming, as Java is a strictly object oriented language).

With the term “Project Initialisation” we refer to the tasks that need to be done(usually done by one person after discussing it with everyone) before the development team

can work on the software code in parallel. Here is a non-exhaustive list of tasks that need to be completed on this phase:

1. Repository creation (Github/Gitlab)
2. Codebase initialisation (setting up the build system, defining versions for the used libraries/frameworks and the programming language, adding code formatters and linters, integrating code quality tools such as SonarQube, setting up Dockerfiles)
3. Architecture Implementation (practically this means writing the classes that will be specialised to implement the functional requirements and creating a coherent filesystem structure for the project's code)
4. CI/CD integration (setting up Github Actions/Gitlab Pipelines that take care of deployments, testing and synchronising the codebase with third parties e.g. library repositories.)
5. Cloud Provider settings + instances that will host the software
6. Domain Modelling for a core set of Entities

The “Implementation” step includes all of the common sub-tasks in software development, such as domain modelling, architecture implementation, testing, documentation, coding. Depending on the novelty of the software being developed this step can be more or less demanding.

The “Evaluation” step involves the installation of the software for the user to be able to access it and the actual usage to determine deficiencies, misunderstandings, bugs and other things that should be fixed before the software is released to production. Without technological sophistication this process involves a representative from the company watching over the customer's usage and discussing potential changes required. This also entails that the informal discussions between the representative and the customer must be documented and converted into actionable items for the development team to fix and integrate into the final release.

The “Deployment” step, traditionally would require specialised personnel (Dev-Ops/System Administrators) and was an operation that could not be done in an automated manner, and for more advanced features, such as DDOS protection, autoscaling, backups would require special treatment and was not something that could just simply be included in the installation but had to be considered throughout the development process.

4.2 Conceptual Approach/Assumptions

Software development projects can be described as the interactions between a business, a customer and the developers(those distinctions can better be understood as roles played by different people, as the developer and the business could be the same person, as is the case in freelance). By understanding this fundamental relationship, we can seek to operationalize the interactions between those primary actors by giving them the tools to cooperate in a straightforward and effective way. The software developed in this thesis mostly addresses the gap in the software business to customer interaction and a bit in the software business to developers' side of the interaction.

While investigating the existing landscape of approaches I came up with the following assumptions/constraints for the solution to the problem of fundamentally changing and automating the software development process.

- **Abstracting Code away is not the answer:** This is evident by the low popularity of NO/LOW Code tools among developers. Also these types of tools tend to be narrowly focused and enforce vendor lock-in (almost unavoidably, the more elaborate their system is, the less it can be mapped to different technologies/vendors) which in the case of most medium-large software projects is not desirable.
- **Software Development processes are deprecated:** Most methodologies that have been developed are before 2005, with the latest being eXtreme Programming(XP) and Scrum frameworks(which we won't talk about in this work). This means that newer technologies have not been integrated into those processes, such as CI/CD, containerization, and IaaS.
- **Transparency is key for long-term change:** The tools supporting this methodology should be completely transparent and modifiable to accommodate the necessary changes for each businesses' needs. The methodology should be clear, and the code generation related process should be clear and open-source.
- **Feedback is crucial when creating software:** Software should not be created without letting the end-users experiment with it and give real feedback. Feedback should be integrated as a core part of the process and the surrounding tools.
- **Software tasks should not be separated in phases:** Common software development tasks such as testing and documentation should not be done

after the code is written. Everything should be done in bundles including testing and documentation. Waterfall has the core problem of separating everything into phases and the many hand-offs between teams. Work should be properly allocated to the respective people (Devs can and should write tests, but QA will push the system to its limit, technical documentation should be written immediately by the devs for features where it is deemed necessary, even if it is just a comment on a file).

- **Everything is converging to code (and Managed Services):** Nowadays, tools are more and more focused on enabling the developer to create, rather than create on behalf of him. Processes and tools should produce machine-readable artefacts in order to advance the goal of automation.
- **We should not discuss processes in a generic manner:** The research until recently has mostly focused on understanding the general method of producing software. To automate processes however, processes should be discussed in the context of a specific Application and Business Domain (industry + software type).
- **Feedback should be specific and not textual and vague:** Feedback from users should be done in a way that pinpoints the exact underlying technical implementation so it can be addressed in a time-efficient manner. For example, when giving feedback on a website it is a different thing to just say “I want this button bigger on that product page” and to say (using a tool of course) “I want the button with text <Submit> on /data?dashboardID=12390123 to be bigger”. The second description does not require a translation from a user feedback text to a technical description (github issue).

4.2 Business Context

It mostly revolves around architectures and software services bought for the implementation of the system. Also budget wise how the situation looks can determine the methodology but not the software itself. Scrum is a methodology that is based on the modularity of releases which means that features can be built depending on the budget available for the project.

When deciding upon technologies, architectures and implementation details, the context of the businesses is implicitly considered. For example, in a big company with many small teams, the microservice architecture makes sense from an organisational standpoint. Scrum makes sense for not too complex projects, where the requirements are not clear from the beginning and with more juniors that need more defined processes in order to learn and understand how software projects work. The business

context is something that influences software development related decisions and should be made explicit and formal in order to allow for further automation of the process. The business context required (meaning it is taken into consideration in decision making) for each type of software is different.

4.3 Requirements Specification

It is important to provide a structured requirement specification in order to be able to achieve a considerable degree of automation. Verbal descriptions are not something that should be completely eliminated but discouraged and only used in uncommon cases to complement the structured information.

Considering that requirements change it is important to address it explicitly. Changes in the requirements can be one of 3 types. It can be adding new desired features, removing existing features and modifying current ones. For the automated approach there should be a way for either to be automatically integrated in the existing codebase, a problem that at its generic form is a research topic in model driven development/architecture. In a structured and controlled environment it can be done up to a degree. By even considering those 3 different scenarios, it allows us to plan ahead for the changes and include them into our process smoothly to save time and generally consume less resources.

4.3.1 Functional Requirements

The main point is to not let the users give us vague answers about what they want that should then be interpreted by the developers and PMs and then inserted into the tasks backlog. We want to restrict their options to what the software platform specifically has and what the domain mostly necessitates. In a website it is evident that the software contains different pages, different UI components and backend functionalities. For more complex software the components are more advanced and should be communicated effectively. Also depending on the software domain the functional requirements can be specified in a more concrete level, directly corresponding to a domain concept, for example in an e-commerce application the customer could choose a loyalty program as a functional requirement.

4.3.2 Non-Functional Requirements

Non-functional requirements (NFRs) can be automated in a more holistic way by considering that different application-domains have different but specific tendencies in non-functional requirements [27].

What this means is that by identifying the application domain, by asking the user for it or by working it out with NLP techniques by the user's description, we can also make an assumption that certain non-functional requirements will be important[citation needed]. By making an informed decision about what the user will want we can guide him further. Also for certain "ities"/"ilities" there are best practices that can be followed. For example, the security in e-commerce websites is very important and is always done in the same manner, regardless of the actual business, which usually involves established practices and technologies such as [OAUTH2](#) with some kind of payment processing platform(e.g. Stripe).

Another way that explicitly specifying non-functional requirements enhances the automation process is that they influence design decisions and enforce constraints in the implementation of different functionalities [28]. This is also demonstrated in [10], where architecture selection depends on the desired non-functional requirements. It can get even more granular as deciding design patterns can be based on NFRs as showcased in [29].

To enhance our process it is important to understand that for different application types, each NFR can be interpreted in a domain-specific way. For example, in a big data system, Performance can be regarded as how fast can Reports and Dashboards be generated, how much time does it take to run certain data pipelines etc. etc.. That way it can be made easier to ask the user for performance related requirements than to vaguely ask him about performance.

4.4 Software Development

Software development is a task that needs to be analysed iteratively into smaller pieces to achieve a satisfactory level of automation. By having a detailed requirements specification and a clear understanding of the business context, decisions can be made faster and even in a partly automated manner.

4.4.1 Software Types/Taxonomies

Understanding the software landscape is important in separating the software automation into sub-projects with different requirements' specifications, constraints, available tools/platforms, quality standards etc. etc. Each software type has a level of difficulty in automating its creation, depending on the various parameters mentioned. In [10], there is a methodology and a tool for selecting between different architectures depending on the Application Type and the Non-Functional requirements. Also in [15]

there is a matrix on the suitability of programming languages for different application domains, based on review of various sources by the authors.

4.4.2 Software Architectures

Software architecture is the basis upon which the actual application is built and considerably influences its Non-Functional properties. When evaluating the business context and the non-functional characteristics we can choose an appropriate software architecture. As far as functional characteristics are concerned any functionality can be implemented using any architectural style(except in maybe extreme scenarios). It is possible to map the combination of a software application type and its non-functional characteristics to a concise evaluation of the different architectures that can be used as previously mentioned and shown in [10].

4.4.3 Component Reusability

To achieve a good level of automation in the development process, it is necessary to have some level of component reusability. In this approach components are considered code that can be modified slightly to achieve our needs(as it is really reductionist to consider completely out of the box reusable components). To talk about re-usability it is necessary to talk about features and how they can be used to automate the development of an application.

In this paper we will focus on the taxonomy of Entity Independent/Dependent Features.

4.4.3.1 Entity Independent Features

Some features do not relate to the objects that are being used, unrelated to the actual domain of the application but are just things that can be added to any system/domain entity. Programmatically this can be understood as something that can be written in a generic way, using the actual domain entity as a parameter to the feature

Examples of this idea are:

- System Observability/Monitoring
- Queueing System
- Reservation System (It is reservation related rules and not that much related to the entity)
- Simulation System
- Search Functionality
- Payment System (it only accepts an Order that has Products with a price)

4.4.3.2 Entity Dependent Features

This is where most of the business logic is found, at least the domain entities part of it. Those cannot be known beforehand and should be written ad-hoc or expressed in some user-friendly language that a non-technical person can understand and write as requirements. It can also be understood as code that only interacts with the domain entities and not external services.

Examples include:

- Applying Discounts to loyal customers
- Determining the amount of books a library member can access depending on his university and his rank.

4.4.4 Domain Models Reusability

Domain Modelling as a task is difficult to automate in a technical/software-based manner as it involves understanding how a domain of human endeavour works. What can be done though is to organise existing knowledge from previous projects and reuse it in projects within the same domain. For example, e-commerce applications do not differ substantially between one another, it is mostly Products in the database, an inventory system and some kind of interface with a payment system/gateway. This means that for common business applications there can be some kind of reusability and not start necessarily from scratch every time, thus saving a considerable amount of time. By creating a partial domain model for common applications, Minimum Viable Products(MVPs) can be created much faster, commencing the feedback loop earlier in the software development life cycle. To achieve this in an organised manner, we can separate domain models into base domain models and domain model extensions.

4.4.4.1 Base Domain Models

Here are the entities that are necessary for a software domain. For example, the class Product is crucial for an e-commerce application and will be present in all related software unavoidably. As such, it can be included in the project initialisation process immediately. For each software domain there is a set of entities that can be included in every related software project, which is important to document in a systematic and language independent way.

4.4.4.2 Domain Model Extensions

In this part are the entities that are optional for a specific domain and complement it in a very narrow and specific manner. Those should be grouped into desired software features and be included only upon customer request. As the knowledge base for a

certain domain expands and more models are included, the greater automation can be achieved, reducing analyst's and developer's man-hours. For example, in e-commerce applications a loyalty program is not absolutely necessary but it is a feature that most businesses would find useful to have.

4.5 Software Development Process Automation Framework

Generally software development methodologies were conceived at times when the technology was less advanced. Automation efforts should consider the technological improvements that have been made and eliminate redundant steps(as discussed in previous sections).

Development automation cannot happen outside the context of a business or a team effort to deliver significant value. When considering tasks that simplify and accelerate the development efforts we should consider the process and organisation within the development occurs. There are existing approaches to software development, from start to finish, which we should understand before we start investing efforts into automating this process.

4.5.1 Analysis and Design Methodology

To automate the software development process it is necessary to have a defined flow of artefacts and steps. To achieve this there should be an analysis step of the Methodology that is about to be automated.

1. **Choose the desired methodology:** The 3 broad categories are heavyweight methodologies, agile methodologies and hybrid methodologies. This decision will be made depending on the business situation/existing business processes.
2. **Analyse the methodology into discrete steps:** While this might seem straightforward, the flow is not always clear in methodologies. For example in the Rational Unified Process the flow is not clearly defined but rather separated into different workflows, while in Waterfall it is obvious what are the steps and in what order they should be done.
3. **Spot repeating steps:** Iterations are really important in software development and must be taken into consideration when trying to automate a process. No software can happen in one go, making it necessary to analyse how iterations happen within a software development process.

4. **Create an iteration model:** It is important to pinpoint where those iterations occur, at what point in time and according to what user input. It can also be understood as “How and when do we receive feedback from the user about the changes he wants in the software?”.
5. **Analyse steps into small, discrete tasks:** As widely known in Software Development, to implement something using computer systems, it should be analysed into small, manageable and simple pieces that can then be implemented. For example, the activity “Receive Feedback” does not contain enough information to be included in an IT system.
6. **For each task define its input and its output:** It is important to model how different tasks communicate with each other through artefacts and how do those get refined as we continue with the business process. This will also help us in the implementation of an automated development process.
7. **Divide tasks into “internal” and “boundary” ones:** The methodology broadly defined, receives some user input and converts it into a representation that can be easily translated into executable code. It is important to understand what lies at the boundaries and what is completely internal. We have actual control within the internal ones and minimal control of the boundary ones.

4.5.2 Implementation of an Automated Process

The implementation depends on the analysis that has been done and requires a structured approach.

1. **Define the User Input method:** This can be done in many different ways and can depend on the type of software being created. There are tools to do so in a generic manner like BPMN, a subset of which can be translated into an executable process as demonstrated in [citation needed]. It is easier to define a different input structure and flow for each type of software and share some common functionality, e.g. asking the user for non-functional requirements.
2. **For each task associate a technology that can automate the transition from input to output:** Different technologies have different strengths and weaknesses and should be used appropriately. For example, converting verbal descriptions into informal requirements can be gracefully done using modern LLM technologies such as ChatGPT, Gemini etc. etc.

3. **The last and final output of the tasks, meaning before the code generation step, should be bridged with the input of the code generation method.** If there is no automatic code generation step and code is written manually, there should be a way to combine the tasks outputs in a way that communicates the requirements concisely to the developers, an approach not recommended for the amount of effort this whole process requires.
4. **Now, having the associations of tasks and technologies, the individual solutions can be independently developed and assessed for their effectiveness and efficiency.** This process is iterative as it might be empirically proven that a technology cannot satisfactorily solve a specific problem and the association should be reconsidered. For tasks that do not have structured data this is not improbable to happen and should be expected.
5. **Plan Iterations based on user feedback.** The most difficult part to automate is iterations based on user feedback. Automatically adding new features and doing modifications to existing code in a predictable manner is not easy and the manual method will probably be preferable.
6. **Integrate the sub-components:** The integration step, depending on the implementation of the sub-components, will vary in difficulty. By integrating those in a single unit of software is essential to streamline the software development process.

4.5.3 Modelling(BPMN) Software Development Methodologies

The methodologies that were modelled here are SCRUM, Waterfall and the spiral methodology. They were chosen as representatives to cover the whole heavyweight to agile spectrum in order to have a more complete understanding of the software process.

4.5.3.1 SCRUM

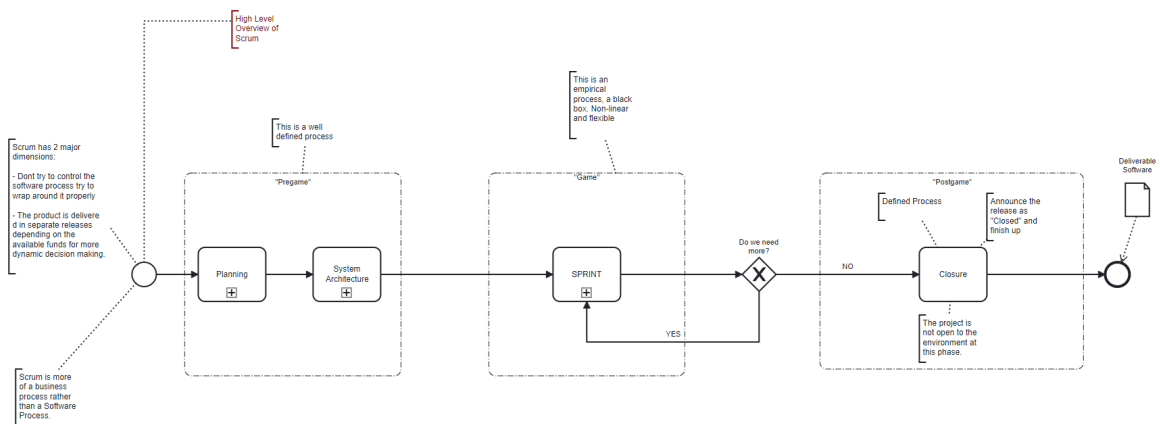


Figure 4.3.a: SCRUM development process-high level overview(BPMN)

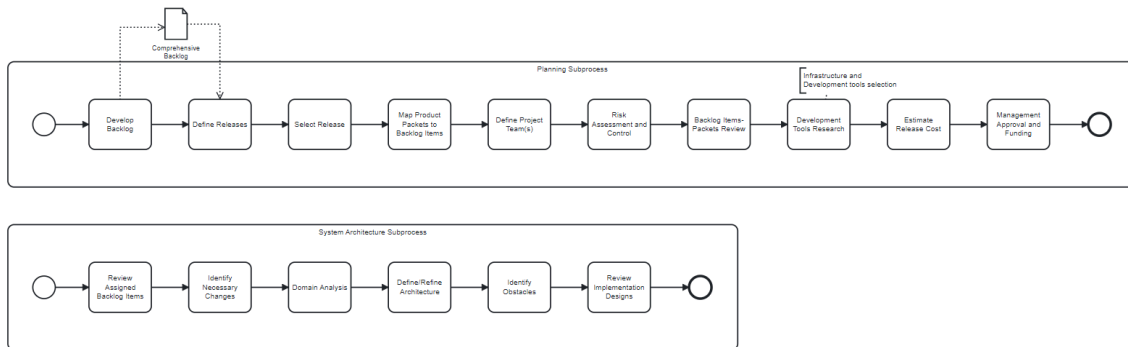


Figure 4.3.b: SCRUM development process-pregame(BPMN)

Towards a domain-specific automated software development process

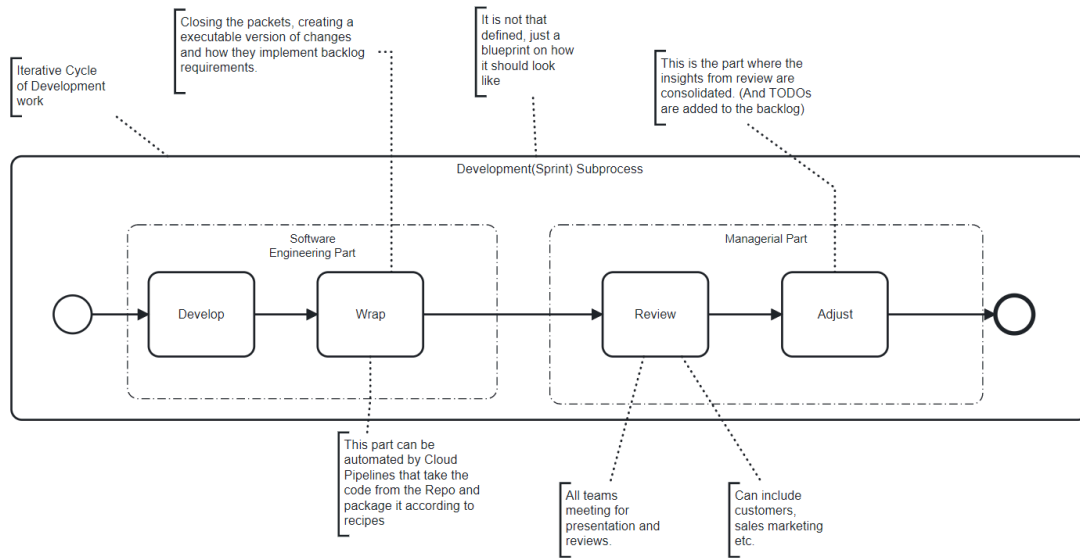


Figure 4.3.c: SCRUM development process-sprint(BPMN)

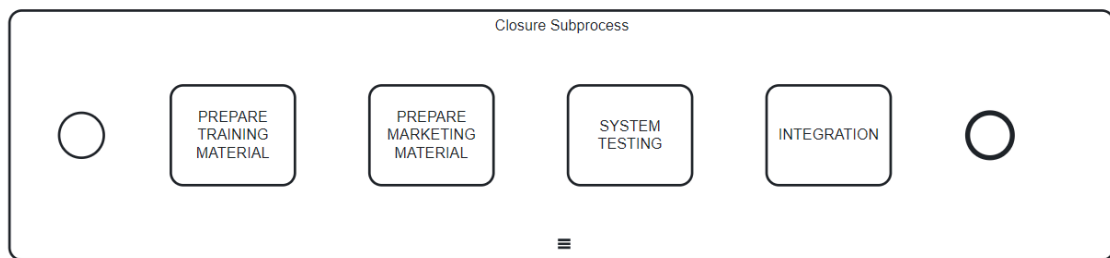


Figure 4.3.d: SCRUM development process-postgame/closure(BPMN)

4.5.3.2 Waterfall

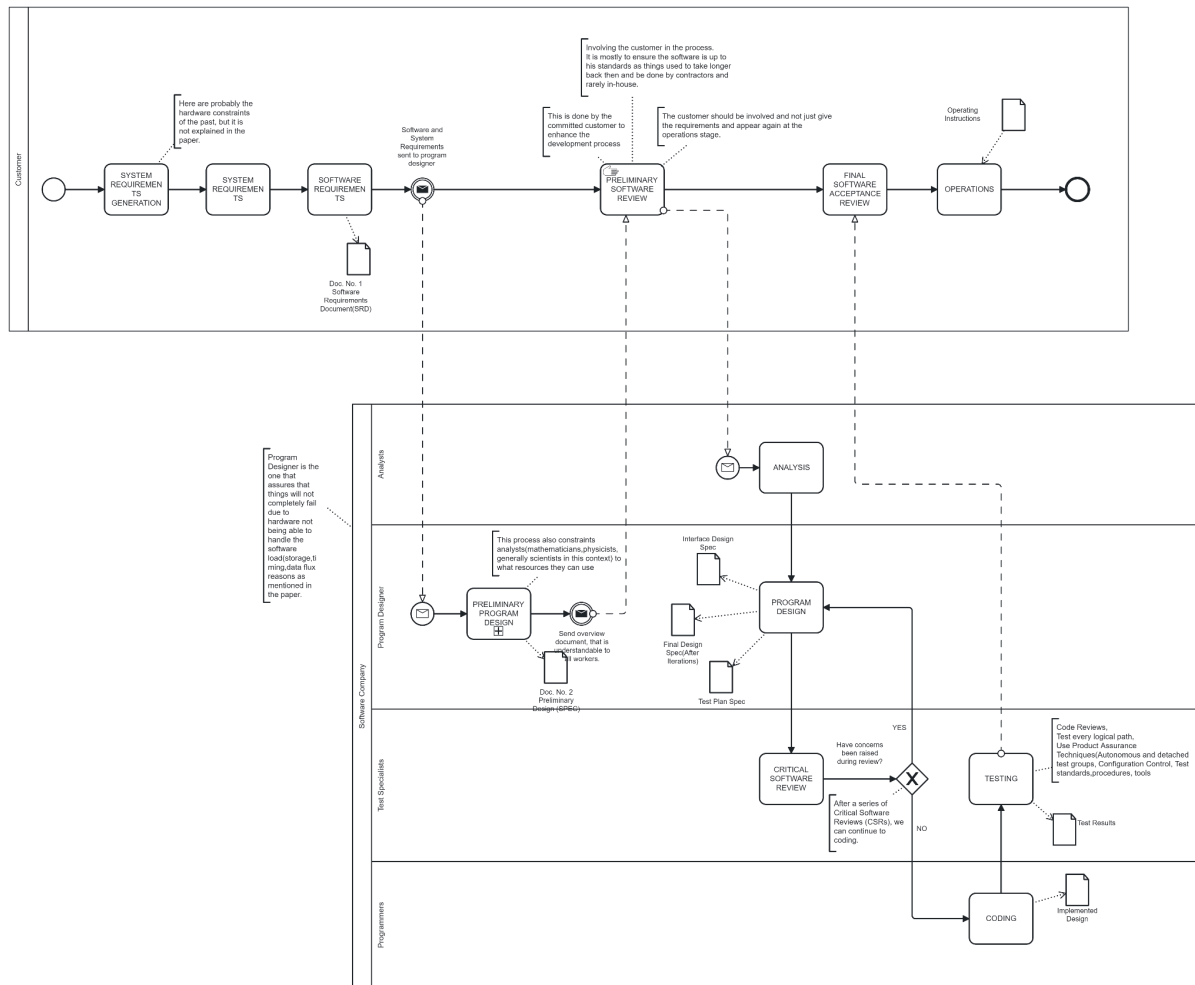


Figure 4.4.a: Waterfall model-Core process (BPMN)

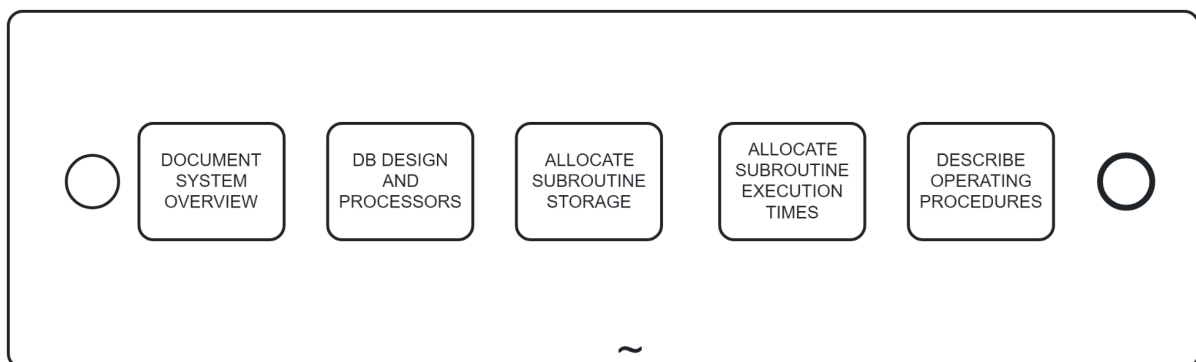


Figure 4.4.b: Waterfall model-Preliminary Program Design (BPMN)

4.5.3.3 Spiral Methodology

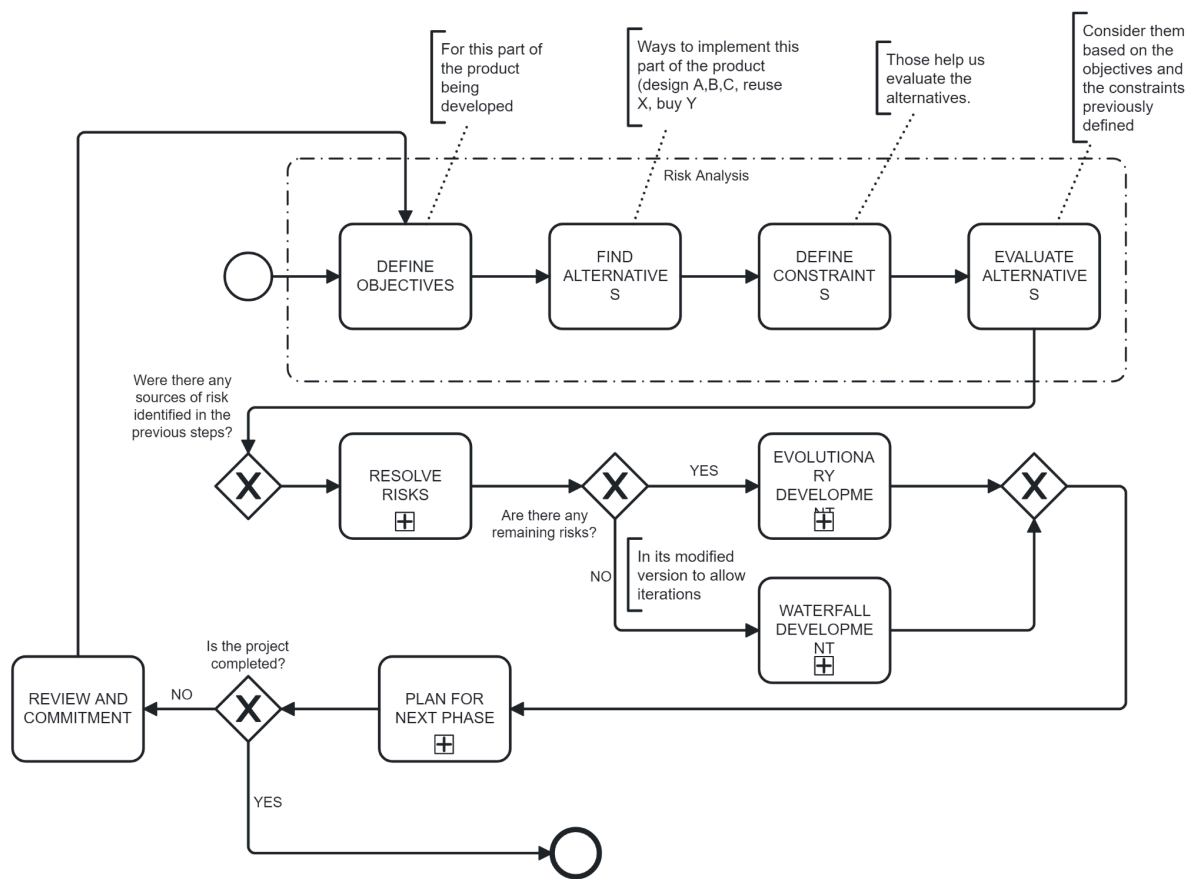


Figure 4.5 Spiral Methodology (BPMN)

4.6 TO-BE Software Development Process

Different steps have different levels of difficulty to automate, but for common software types where the non-functional requirements do not necessitate an overly rigorous and scientific approach (like simple e-shops, blogs, dashboards etc. etc.) automation is straightforward.

In this one-dimensional flow (iterations will not be discussed but they can easily be accommodated in the existing framework by inserting feedback breakpoints) what needs to be emphasised is that each step has a cost (mostly man-hours) and that technologies and research methodologies exist to automate various sub-steps of each high level step. A major thing to note here is that now the roles/participants have been reduced to only 2, the customer and the development team. It will be explained as we progress through the individual steps.

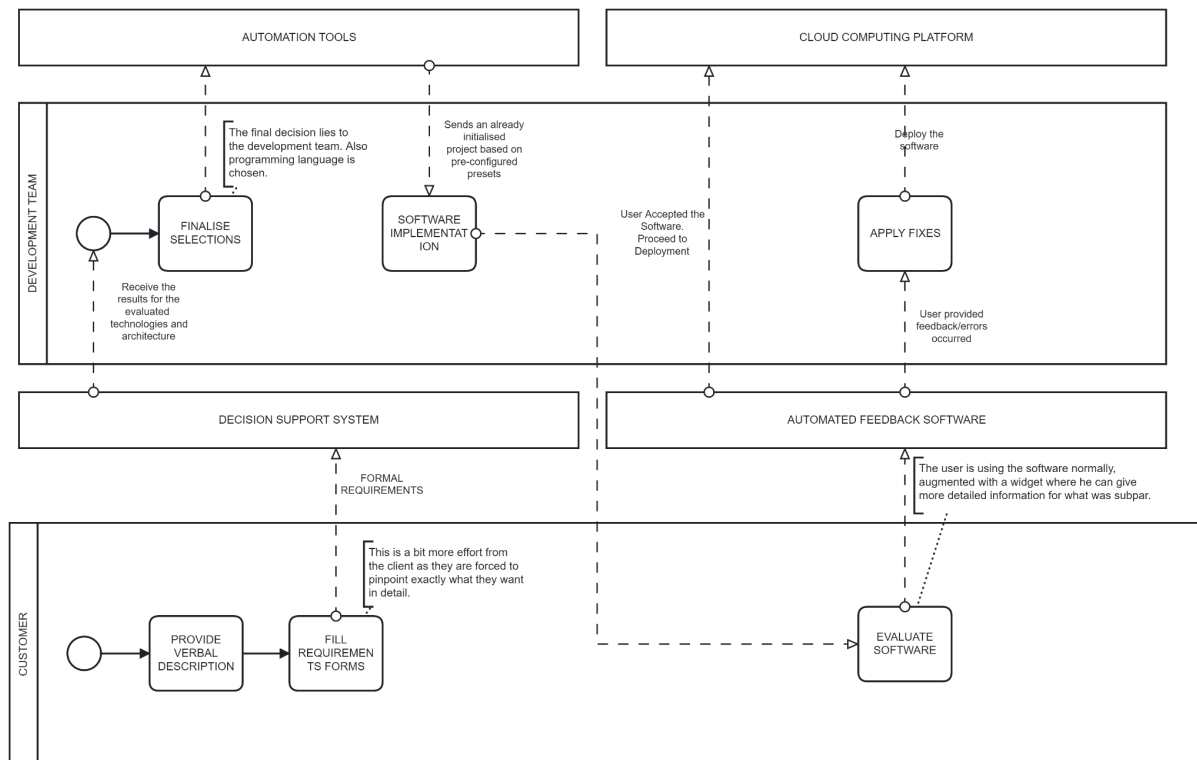


Figure 4.6: Modernised Version of Software Development process flow(BPMN)

In this updated version the customer does not interact directly with a representative but through a software that is specifically designed to elicit requirements in a structured way so they can be utilised in future automations, while also giving the user the capability to provide some verbal descriptions that can be used to understand his situation. The experience can be streamlined, by utilising LLM techniques e.g. by pre-filling some requirements forms from the verbal descriptions, by helping him choose between alternatives and by giving hints and explanations when the user feels stuck. Also an LLM can be used to translate verbal descriptions into concise summaries relevant to the creation of the software. This whole approach is beneficial in many ways:

1. A richer amount of information can be extracted
2. The meeting of 2 people can be reduced to 1 person interacting with a software.
3. By controlling the shape of the user input, we can ensure that the structure of information matches what is useful in the software development process.
4. An efficient communication channel is established, as the software that receives the requirements also notifies the development team.

5. By controlling the structure of requirements for each software domain, we eliminate the need for a dedicated analyst/domain expert, as their knowledge is embedded in that same structure.

At this point the Decision Support System, resembling that of [13], [14], receives the structured information, saves them for future use by different systems, and uses them to produce a report containing evaluations of the different architectures and the different technologies involved. This document will be used by the developers to finalise the selection of technologies(RDBMS,Cloud Provider etc. etc.). This automates the comparison process which is unjustifiably time-consuming and taxing on developers.

When the developers finalise the bundle of technologies, this decision is sent to the automation tools system where the project is initialised according to a pre-made configuration. This is where the senior developer/technical lead would go ahead and initialise the project as outlined in the AS-IS process. By embedding their knowledge into a configuration they are involved minimally throughout the project, only for more technical management purposes(code reviews, mentoring etc. etc.) and less for actually writing the code. It is an important part as this reduces the man-hours required as for such roles the compensation can get quite high. “Automation Tools” is an umbrella term that includes a variety of software that assists in the creation of a project(by adopting the principles presented in the proposed approach) that has already incorporated the repetitive and already known parts of the implementation, an example being interaction with 3rd party APIs(using the OAS in combination with code generators as explained in Chapter 5).

The implementation step is entirely dependent on how detailed are the functional requirements provided by the customer. The more structured they are the higher the automation potential as will be showcased in the software implementation of this approach. Implementation at 100% is not doable as the amount of edge cases where a developer’s attention is required is non-negligible. Automation in this space mostly revolves around code reusability from similar projects and pre-existing domain models from similar projects. For specific commonly expected features, automation can be achieved by the appropriate templates and code generation processes.

For the “Evaluate Software” step the only automation that can and should be done is by embedding feedback software inside the app so the user can provide his feedback in an efficient manner by localising it and communicating this information straight to

the developer. For example, for websites a feedback widget could be implemented where a user navigates in each page and can submit complaints like “Data loading is too slow”, “I can’t find X”, “This button should say Y”, “The layout is counterintuitive” etc. etc. and have them immediately post them as Github issues. This way, back-and-forth communication is reduced to the absolute minimum, the customer using the software and submitting the change requests for the developers to implement.

The “Deployment” step shown in AS-IS, is missing from TO-BE, as it is a technical problem and thus is already automated for trivial cases. By adding proper CI/CD processes and integrating cloud computing, deployment occurs automatically depending on the setup done in the project’s initialisation. For non-trivial cases, it is required to have a more disciplined approach in the consideration and implementation of architectural patterns and utilise developer man-hours in order to customise the deployment environment, e.g. deploying on-premises due to strict regulation.

Chapter 5: Concrete Methodology for Data Services

In this section we will analyse and design a tool that will automate the development of Big Data Systems, more specifically, systems that gather data from various sources(APIs), with a considerable volume of data and are often accompanied by Dashboards to enhance decision making capabilities. Its architecture can be summed up in the following figure:

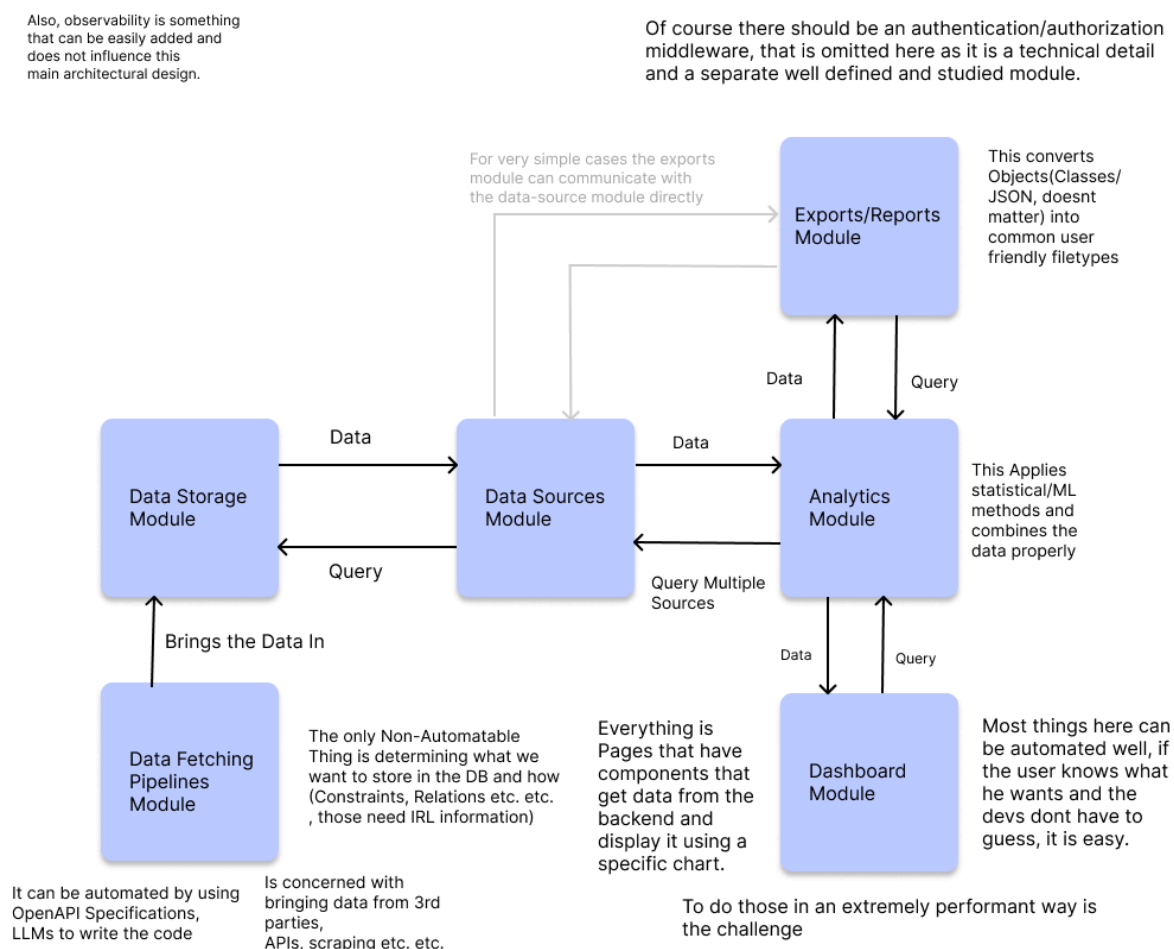


Figure 5.1: Big Data Systems high level overview

This model is a version inspired by the one Microsoft uses in Azure and someone can read further here [\[article\]](#).

We will now analyse the implementation of each module to understand where automation can fit in and in what ways.

5.1 Data Sources Automation

To use data sources in the software the appropriate access to third-parties' data is needed. To do so in an architecturally sound way, libraries can be used that give us access to those sources in a programmatic way (SDK) so we can build our pipelines. As API technologies have matured considerably there are plenty of ways one can go to automating the consumption of a 3rd Party API depending on what it provides for people to adopt it (in this paper web scraping will not be discussed). If there is an available OAS document this can be done by using code generator tools previously discussed, and filtering out undesired functionalities. In the case that an OAS document is not provided, more advanced techniques should be used in order to extract the relevant information from traditional API documentation pages where equivalent information is provided, but in a non machine readable format.

For each of the data sources we can consider the following to be the minimum necessary non-technical information to capture the requirements from the customer.

- The update frequency (fetch that data every X time, DAILY-HOURLY-MONTHLY etc. etc.)
- How many data should be at once in the DB (e.g. we should always keep 10 years back of data in the DB)
- The time range that we want the actual data for (e.g 2005-2015)

5.2 Dashboards Automation

The high-level architecture of the UI layer(Dashboards) of big data systems can be described as a combination of the following elements.

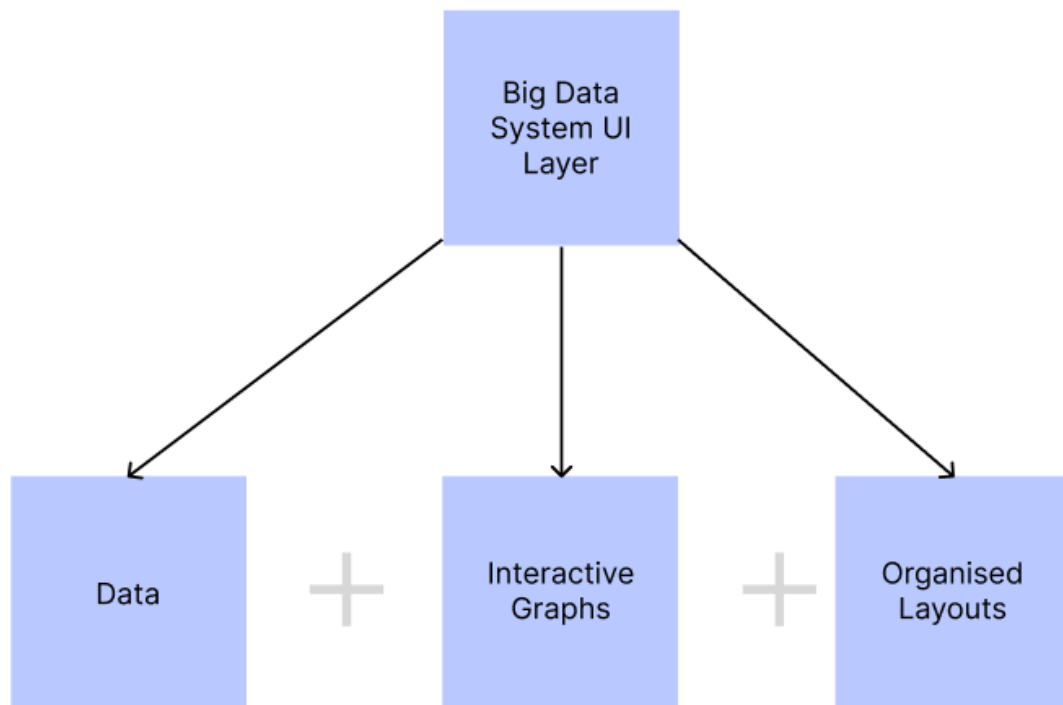


Figure 5.2: UI layer in Big Data Systems

For each dashboard page a layout can be defined which can be greatly automated by using tools that enable the user to specify in great detail his desired layout. In each part of the layout an interactive graph can be fit into where the data is fetched from the sources provided by the API. Each graph can use data from multiple sources and combine them into a single UI component.

Presentation types can be separated into 3 displayed things:

- Realtime Data(this labelling can either be done by an AI or manually by the user)
- Historical Data(this labelling can either be done by an AI or manually by the user)
- Metrics and Statistical Measures of those

The distinction between realtime and historical data also makes sense from a technical standpoint as the underlying technologies can differ(e.g. Websockets vs Refetching through HTTP).

5.3 Analytics Module Automation

Analytics can be understood at a high-level as applying statistical methods to data. This is the most difficult part to really automate as selecting the effective statistical analysis tools requires an understanding of the data at hand beforehand which is difficult to do and AI/LLMs are not yet particularly suitable for. What can be done is to provide a shortcut that connects the statistical analysis to the data in an automated manner and generates the related code, while also making use of specific libraries, like [Spark](#). The user input required can be summarised as a list of associations between bundles of data and the respective statistical method to apply.

5.4 Export/Reports Module

This part is responsible for producing files in formats that can be shared among stakeholders(excel spreadsheets, pdfs, powerpoints etc. etc.). This is automatable for a wide variety of cases, libraries for converting objects to filetypes or template-based preparation of exports. It can also export the default dashboards presentation as a file. The structure of user input can be explained as a list of associations between either unprocessed data from the data sources module or processed data from the analytics module and the desired filetype(.pdf,.csv,.docx etc. etc.).

5.5 Data Storage Module

The data storage module is responsible for the persistence of data. There are many different approaches to data persistence such as SQL,NOSQL,Graph DBs. There is a significant amount of different tools that implement those approaches with minimal differences on very specific features offered. What matters most is the type/technology of storage we need and not a specific tool(SQL,NOSQL,GraphQL,Time Series DB, Graph DB, AI stuff Vector DB etc. etc.). Choosing between database systems in an automated manner has been demonstrated in [dss system]

A non exhaustive list of parameters to be considered in that decision are:

- The size of the data (in orders of magnitude not a specific number)
- Its shape(is it static or dynamic?)
- Developers' knowledge
- Unique features(graph db, time series etc. etc.)
- Non-Functional requirements
 - performance

- cost
- complexity of setup
- availability
- reliability
- Features

The choice of database is also related to the choice of cloud provider, though not directly dependent on it. All cloud providers provide a managed database service and there are some popular managed database services that are provided outside of the cloud provider(e.g. [DigitalOcean](#))

5.6 Functional Specification

From the analysis in previous paragraphs, the input required can be concisely summarised as follows(through industry experience this list can be enriched with more fields):

- Data Sources Module
 - For each data source we need this information:
 - Frequency (Fetch it every X time)
 - Duration in DB
 - Data Time Range
 - Update Rate
 - Realtime/Historic
 - Validation Rules (what is allowed and makes sense in our case?)
 - Filtering Rules (what makes sense but we dont want to put it in our DB)
- Dashboards Module
 - Page Layout
 - Components(Metrics/Chart)
 - Associate components with charts and data sources
 - For charts what interactivity is required?
 - Customizability Features(Adding/Removing Stuff, Moving Charts around, Changing Colors)
- Export/Reports Module
 - Define Reports
 - For each report
 - Define file type
 - Define things to include(charts/metrics/plain data)
 - Produce on demand or Just produce daily reports for people to view?

- Data Storage Module
 - Shape of Data(classes/objects)
 - Relationships/Constraints (This is difficult for a user/customer to define in a complete manner, so simple descriptions can be used)
 - Any Special Feature required? (e.g. Graphs,Time Series etc. etc.)
 - How often should backups be done for each data source?
- Analytics Module
 - What machine learning methods should be applied, using which data?
 - What metrics should be calculated, using which data? Describe Calculation/Use existing formulas

5.7 Non-Functional Specification

- Data Sources Module
 - How long it takes to fill up the DB with the data we want?(Performance)
- Data Storage Module
 - How long should queries take?
 - How much storage should be used, expressed in cost terms? (the corresponding cloud storage cost)
 - Go for a more complex solution or something simpler?
 - Availability? (99.999 % or lower would also be okay?)
 - Reliability?
- Dashboard Module
 - Should it be customisable or static?(Adding Removing Charts, Moving them around etc. etc.)
- Export/Reports Module
 - How long should it take for each report to be produced? Is that critical?

5.8 Business Context Required

The business context will influence the implementation of the presented system structure. In big data systems the most important decisions revolve around the architecture used and the data storage technology utilised. The business context required here is:

- Budget for storage: As it will be the driving cost of the application it is important to understand the businesses' capacity to invest in infrastructure
- Development Teams Size and structure: How many people will be involved in the creation of the system

Budget for storage is important for a variety of decisions that should be made down the road. For example, how many backups are financially viable, what volume of data will actually be kept in the database and how often data will be deleted. Also technologies and tools differ in their storage efficiency so in most cases this constraints our selection further and facilitates the respective decision making process. Cost is also important when choosing between different managed database providers as pricing plans differ significantly.

The size and structure of development teams mostly influence the architecture implementation. Microservice Architecture makes sense when there are multiple teams working on the same project but increases significantly the complexity of the system, which would be unjustifiable in a small team.

Chapter 6: InfoDev, Automation Component for Data Services

Following the principles and ideas previously discussed, an implementation for a component in the whole automated process was done. The general idea is to guide the user through simple steps and intuitive AI enabled UI/UX to complete the requirements process in an efficient manner. In the first screen the user can select the type of software he wants to create, in future versions this is something that could be automatically picked by providing a general description of what he wants and using common AI methods to infer the type of software he is looking for. Based on the results in [12], it could be said that most functional requirements for a specific software domain mostly concentrate in a few categories and that's why this step is included here and is the first one. By understanding a software domain in depth, automations can be created for the most common requirements types.

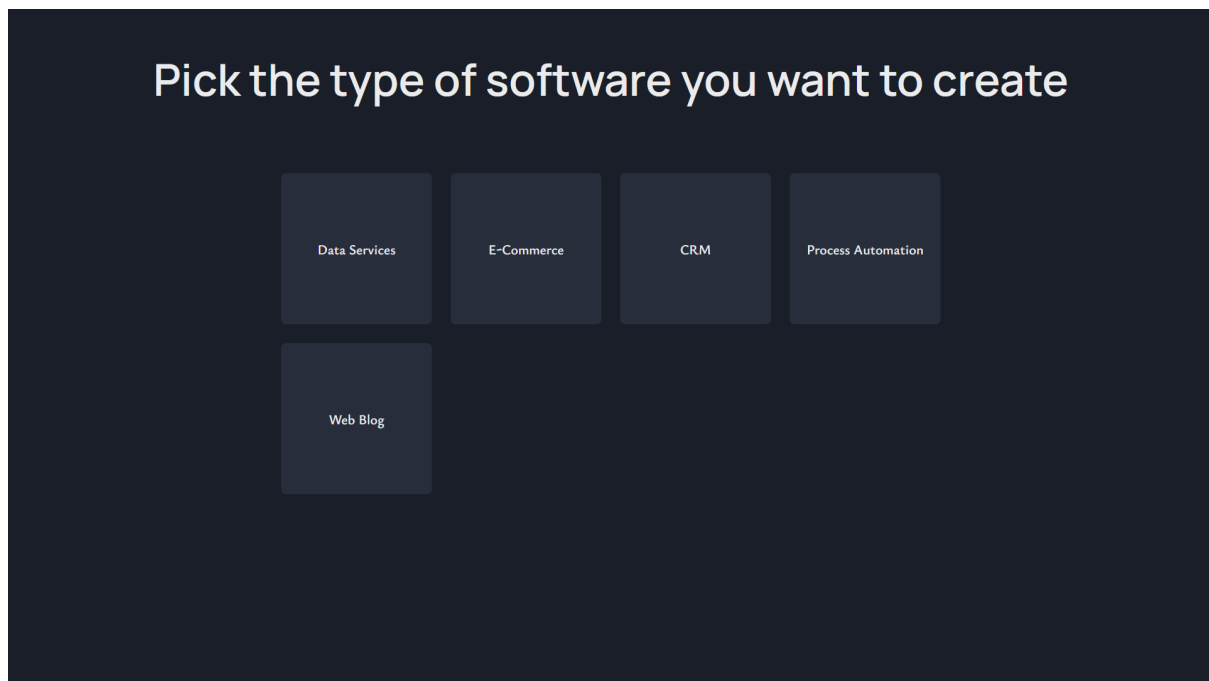
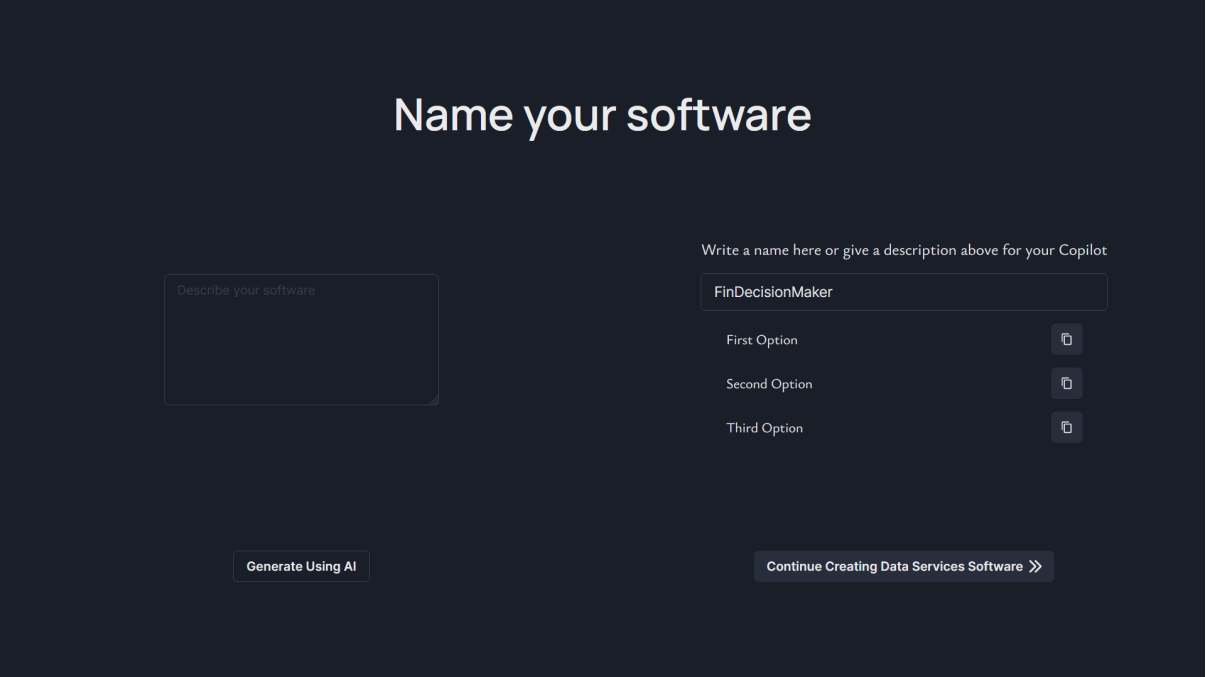


Figure 6.1: Home Page of InfoDev

In this screen the user provides a name of the software which will be used later to create the cloud infrastructure(the github repository in our case, although more things

Towards a domain-specific automated software development process

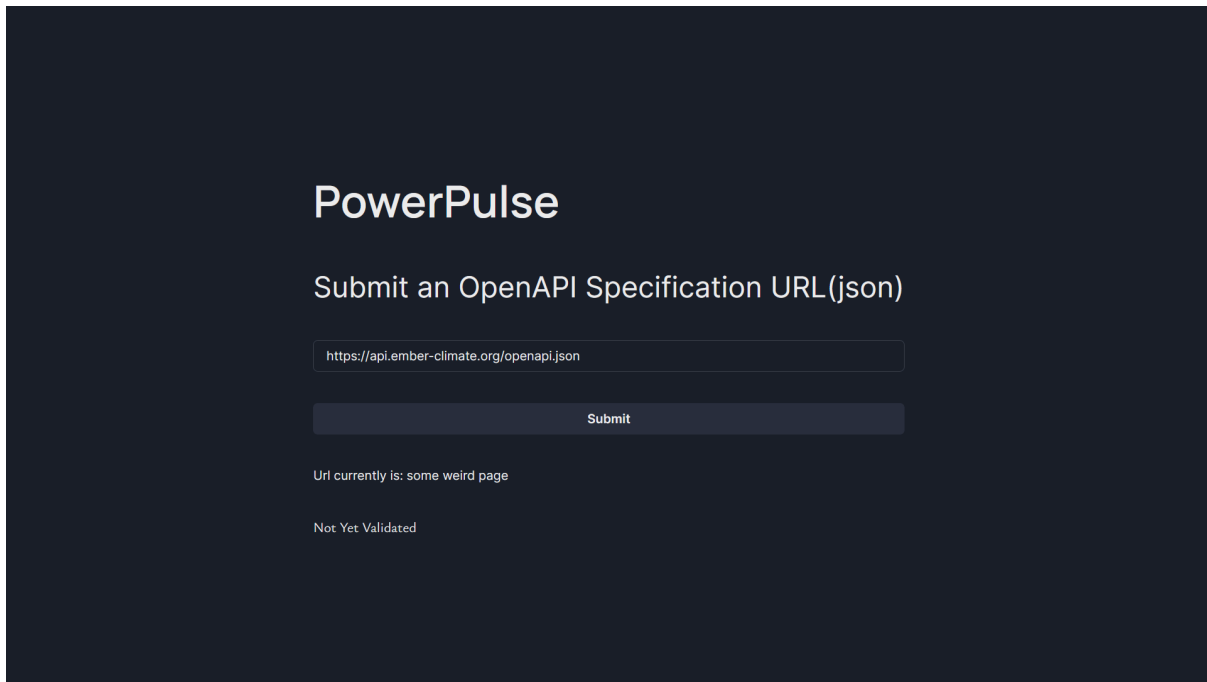
could be added). Also the user has the option to use AI to help him out with names, as it is understood that naming things is hard and would be an obstacle for users.



The screenshot shows a dark-themed web interface titled "Name your software". On the left, there is a large text input field with the placeholder "Describe your software". On the right, there is a smaller text input field with the placeholder "Write a name here or give a description above for your Copilot". This field contains the text "FinDecisionMaker". Below this field are three rows, each with a label ("First Option", "Second Option", "Third Option") and a small icon of a document with a checkmark. At the bottom left, there is a button labeled "Generate Using AI". At the bottom right, there is a button labeled "Continue Creating Data Services Software >>".

Figure 6.2: User names the software to be created in InfoDev

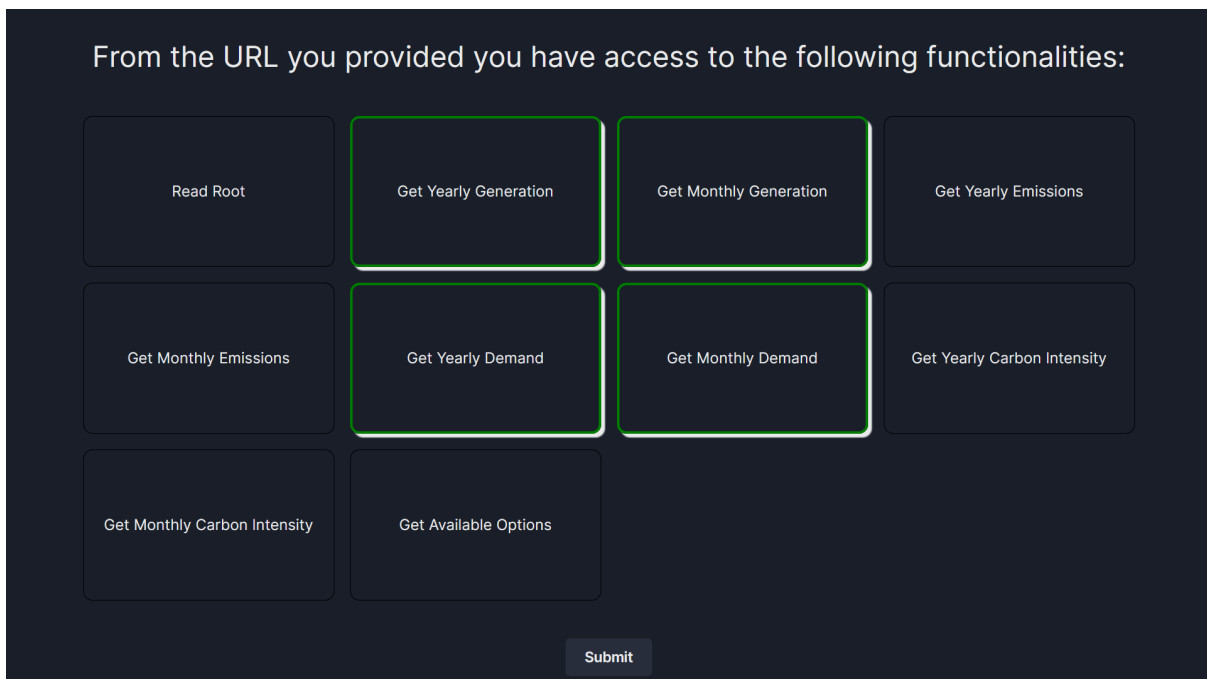
Now the user should be able to submit an OAS url in order to import the data from the third-party he wants(as Data Services software mostly revolves around creating a DB with complete data from various sources). This step has some major limitations as this is currently an MVP for demonstration purposes which should be discussed. The OpenAPI Specification is a technical detail and the user should not be exposed to unnecessary complexities and technicalities. To tackle this issue, in future versions of the software data from the [APIGuru](#) platform will be integrated to enable a search functionality for the keywords the user provides. Another major problem here is the inability to import data from multiple sources and not just one, this is also a feature that will be addressed in future development efforts.



The image shows a web interface for 'PowerPulse'. At the top, the title 'PowerPulse' is displayed in a large, white, sans-serif font. Below it, the instruction 'Submit an OpenAPI Specification URL(json)' is shown in a smaller white font. A text input field contains the URL 'https://api.ember-climate.org/openapi.json'. Below the input field is a dark grey button labeled 'Submit'. Underneath the button, two lines of feedback text are visible: 'Url currently is: some weird page' and 'Not Yet Validated'.

Figure 6.3: User submission of data source(OpenAPI Specification)

Now the user can select among the different API Operations provided by the input URL. This is the part where automation is non-existent as the user should have the freedom to select specifically what he wants in his system.



The image shows a web interface for selecting API functionalities. At the top, the text 'From the URL you provided you have access to the following functionalities:' is displayed in a white font. Below this text is a grid of ten buttons, each representing a different API operation. The buttons are arranged in three rows: the first row has four buttons, the second row has four buttons, and the third row has two buttons. The buttons are labeled: 'Read Root', 'Get Yearly Generation', 'Get Monthly Generation', 'Get Yearly Emissions', 'Get Monthly Emissions', 'Get Yearly Demand', 'Get Monthly Demand', 'Get Yearly Carbon Intensity', 'Get Monthly Carbon Intensity', and 'Get Available Options'. The buttons 'Get Yearly Generation', 'Get Monthly Generation', 'Get Yearly Demand', and 'Get Monthly Demand' are highlighted with a green border. At the bottom center of the grid is a dark grey button labeled 'Submit'.

Figure 6.4: Data selection from third party system

Next up is the metadata for the Data Sources. By embracing the principle of “Don’t let the user give vague answers for what he wants,give him only concrete options” we provide an interface where the user can provide us the following information:

- The frequency of the information at hand(Hourly,Daily,Weekly,Monthly,Quarterly,Yearly)
- The time range he wishes to store in the database
- And a distinction for whether the data is real time or historical, this is a distinction required for technical purposes as those differ in implementation details. This information is also pre-filled by AI for a frictionless experience.

Set Metadata for Data Sources

Available Data:

- Get Yearly Generation
- Get Monthly Generation**
- Get Yearly Emissions
- Get Monthly Emissions
- Get Yearly Demand
- Get Monthly Demand

Get Monthly Generation

Monthly

From Date: 06 / 29 / 2024

To Date: 06 / 29 / 2026

Historical

< Previous Next >

Submit

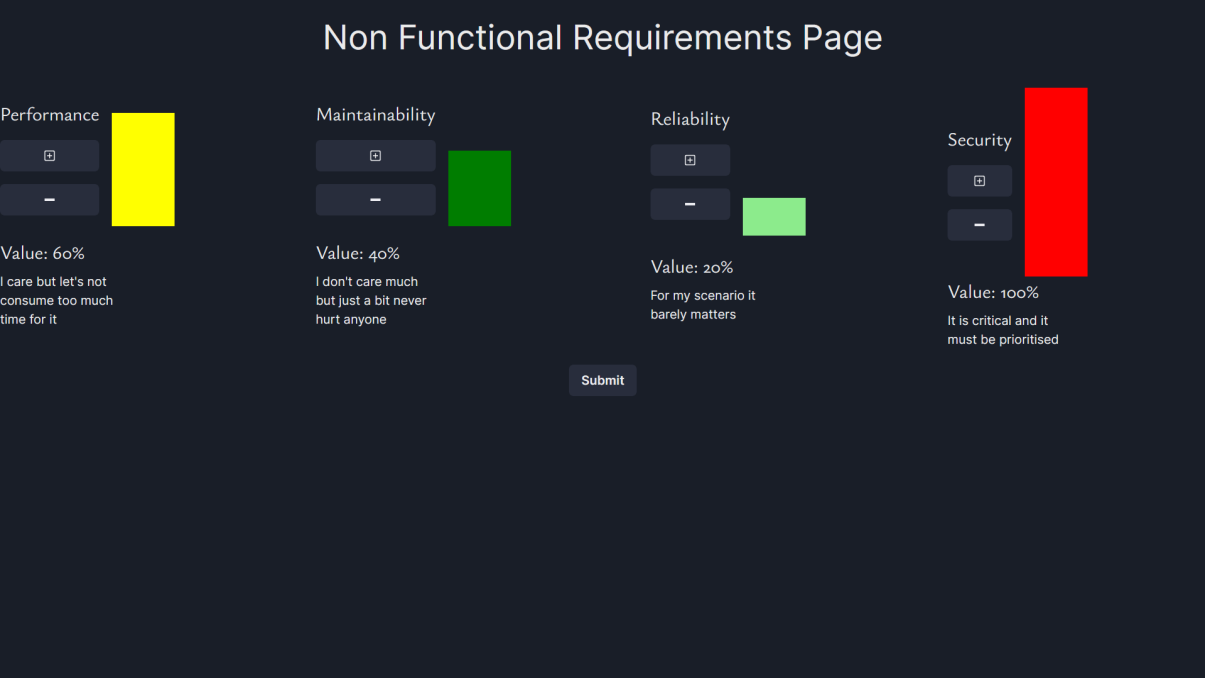
Figure 6.5: Metadata for data sources page

At this point the user can create his own page by adding charts in different rows. While it may seem a trivial addition to the automation effort, UI is the only thing the user actually perceives so it is important that he chooses how he expects it in an exact manner as back and forths cost developer-hours and should be avoided. Here a simple layout creator is provided where the user can add and resize charts in rows and columns. Anything more complicated is unnecessary.



Figure 6.6: Layout creation page

And the final step before getting to the automation/codegen stage is pinpointing the exact non-functional requirements of the system. Here a few basic ones were chosen, this is not an exhaustive list, but in future versions the ISO 25010 could be consulted in order to determine the relevant ones. Also, depending on the software type, different requirements could be prioritised. By having explicit non-functional requirements, we can use this information to evaluate different architectures in an automated manner, as done in [10].



The image shows a web interface titled "Non Functional Requirements Page". It contains four columns, each representing a different non-functional requirement: Performance, Maintainability, Reliability, and Security. Each column has a slider control with a colored bar indicating the selected value. The Performance slider is set to 60% (yellow bar), Maintainability to 40% (green bar), Reliability to 20% (light green bar), and Security to 100% (red bar). Below each slider, there is a text description of the value. A "Submit" button is located at the bottom center of the page.

Requirement	Value	Description
Performance	60%	I care but let's not consume too much time for it
Maintainability	40%	I don't care much but just a bit never hurt anyone
Reliability	20%	For my scenario it barely matters
Security	100%	It is critical and it must be prioritised

Figure 6.7: Non Functional Requirements page

By clicking submit we are given two different options: “Generate Document” and “Generate Code”. Generating the document will produce a Markdown file(a file format popular to programmers, and supported by Github), with all the information provided in last steps. As this is not the end goal for this software, generating the document will be demonstrated in the case study.

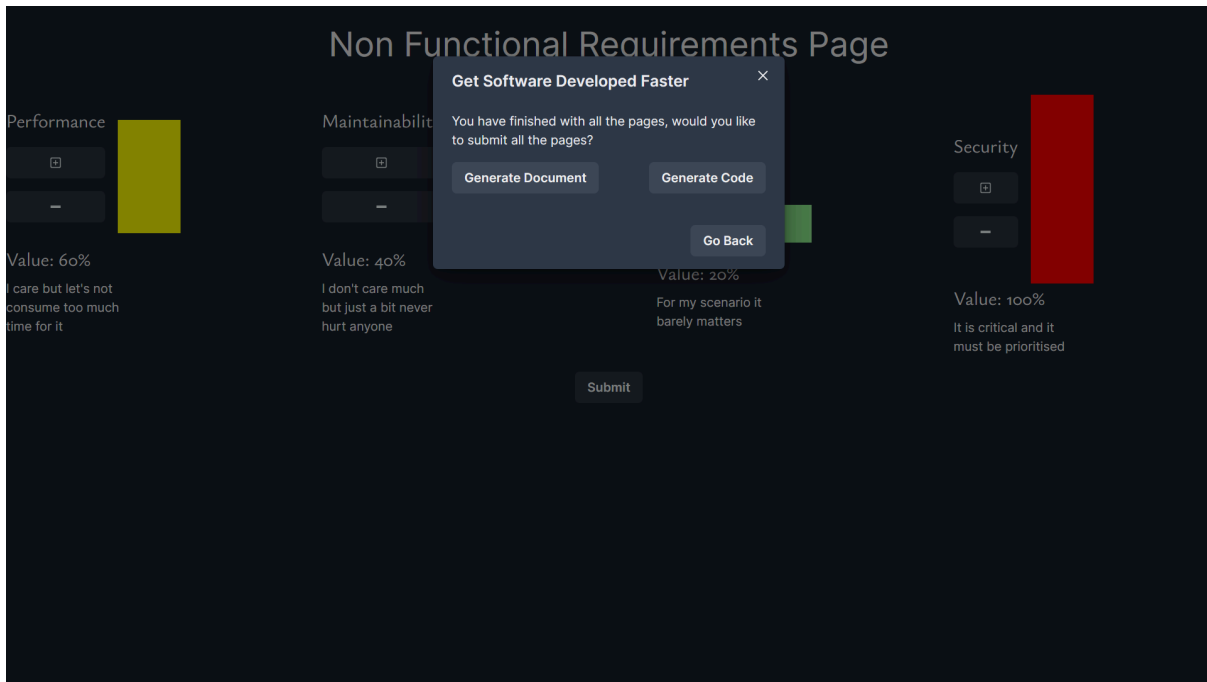


Figure 6.8: Modal for user action based on the submitted input

By clicking “Generate Code” we are taken to the very last page of the software where the code generation will start. Here the user can choose the programming language and then generate the “developer blueprints”, a user friendly term for github issues and boilerplate code. This step is clearly wrong methodologically as the end-user should not be deciding the technical tools used by the development team. It was only included for demonstration purposes and due to the limitations of a functioning MVP. Ideally a technical configuration shall be provided by the development team.

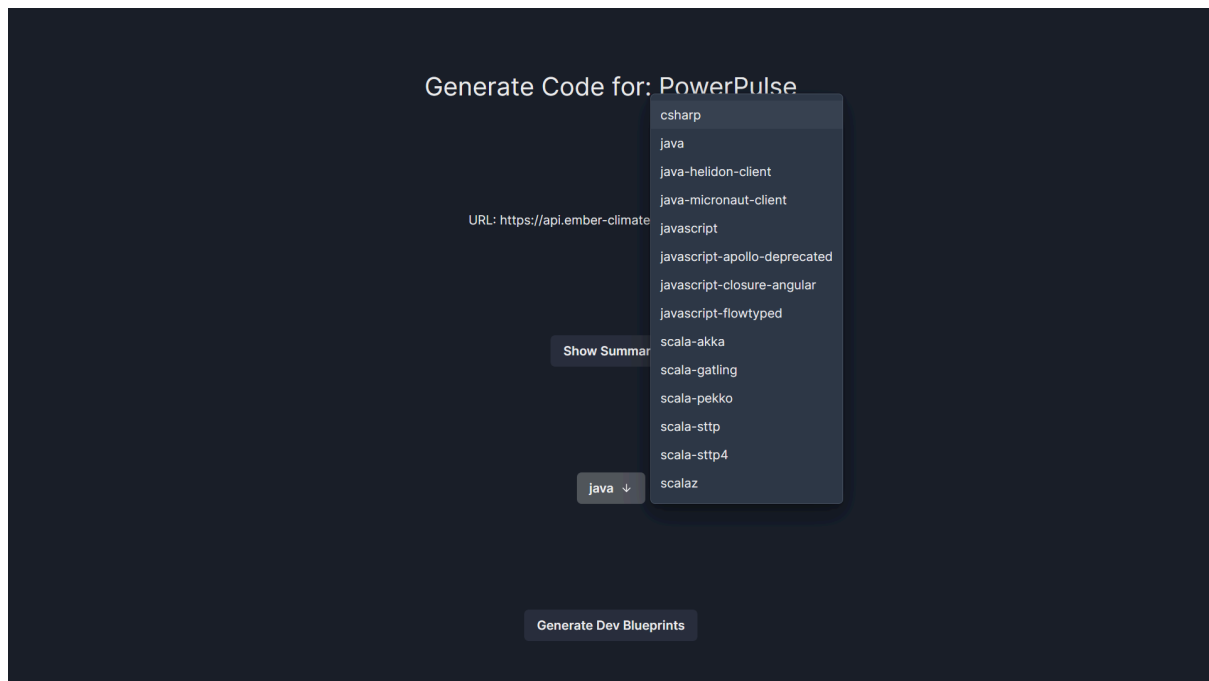


Figure 6.9: Code generation page in selected language

By clicking “Generate Dev Blueprints” now the automation begins. The InfoDev software does 4 major things to shorten development time:

- Create the Github Repository(and provide an LLM generated description of it)
- Create issues for the pages the user wants to implement so it can be assigned to specific developers.
- Generate the boilerplate client code(in the backend’s language) to consume the API provided, so the information can be stored in the database.
- Generate the boilerplate frontend code(by using a common React.js template) and some custom code for the Pages(one file per page).

All of this saves costly developer man-hours and is ideal for small projects where the requirements are clear and it is not a novel type of software.

Chapter 7: Case Study on Energy Data for Energy Related Forecasting

7.1 Scenario Outline

To demonstrate the effectiveness of the Automaton Software let's consider the following scenario. The case study is for an imaginary company EnerFinance, that is occupied with trading on energy markets. To make more data-driven decisions, a large amount of data is required, presented in a simple and concise manner in charts. One of the main data required to determine the market's tendencies are how much energy is generated at all times, and how much energy is demanded/consumed at all times. What should be made clear is that requirements can be captured utilising intuitive software and that structured input can be used to greatly reduce development efforts.

7.2 InfoDev Demonstration

At first the customer, EnerFinance should pick the type of software that is needed. In our case a data services software is ideal.

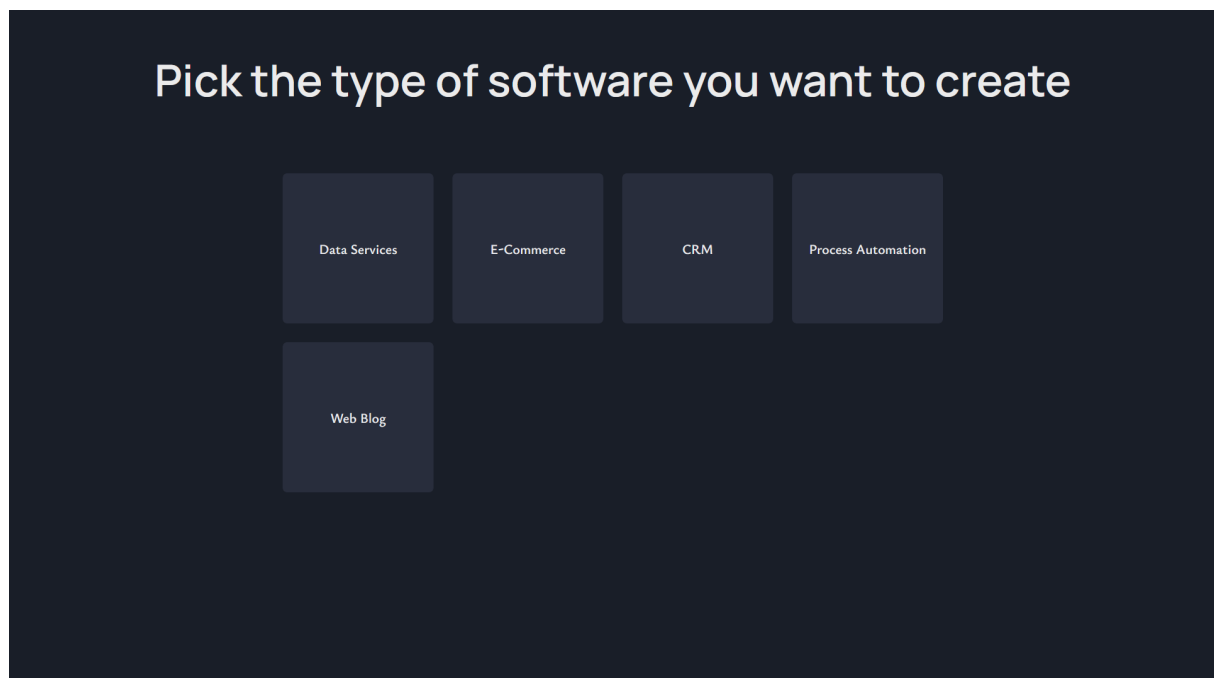
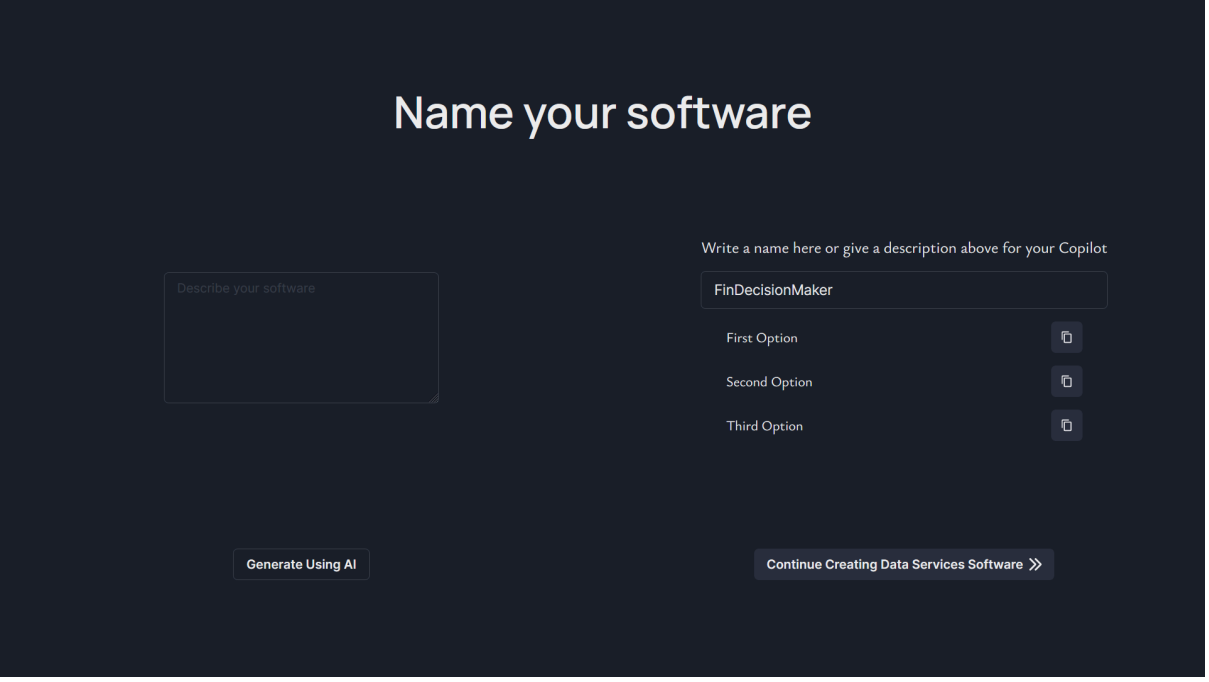


Figure 7.1: Step 1: Select software type

Towards a domain-specific automated software development process

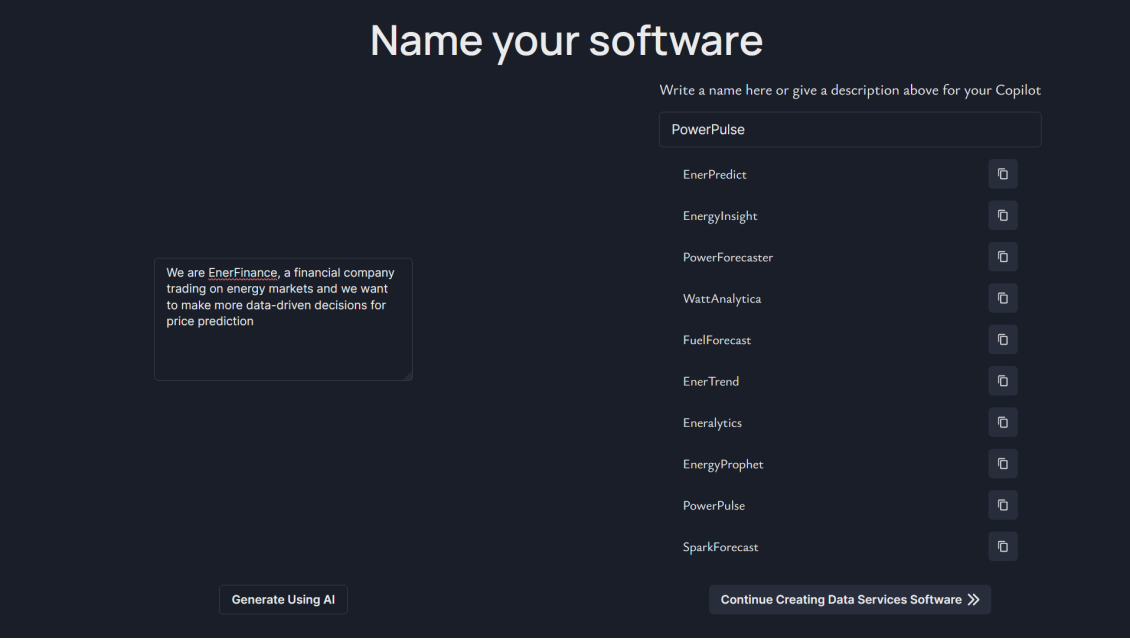
Now a name should be given to the software to be created. By providing a simple description of our case we can get some AI generated name suggestions.



The screenshot shows a dark-themed web interface titled "Name your software". On the left, there is a text area labeled "Describe your software" which is currently empty. On the right, there is a prompt "Write a name here or give a description above for your Copilot" followed by a text input field containing "FinDecisionMaker". Below this input field, there are three rows, each with a label ("First Option", "Second Option", "Third Option") and a small square icon with a right-pointing arrow. At the bottom of the interface, there are two buttons: "Generate Using AI" on the left and "Continue Creating Data Services Software >>" on the right.

Figure 7.2: Step 2: Submit software name

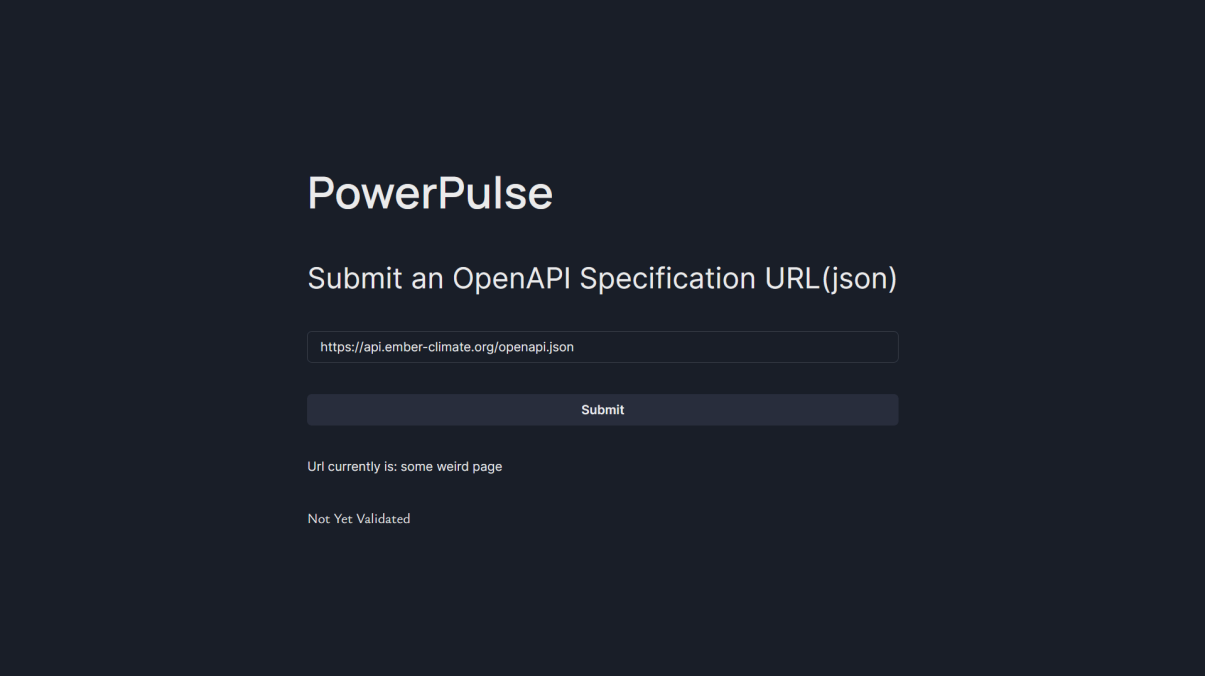
We are choosing PowerPulse, we could also submit a custom name.



The screenshot shows the same "Name your software" interface. On the left, the text area now contains the text: "We are EnerFinance, a financial company trading on energy markets and we want to make more data-driven decisions for price prediction". On the right, the text input field now contains "PowerPulse". Below the input field, there is a list of ten AI-generated suggestions, each with a small square icon with a right-pointing arrow: "EnerPredict", "EnergyInsight", "PowerForecaster", "WattAnalytica", "FuelForecast", "EnerTrend", "Eneralytics", "EnergyProphet", "PowerPulse", and "SparkForecast". At the bottom, the same two buttons are present: "Generate Using AI" and "Continue Creating Data Services Software >>".

Figure 7.3: Step 3: Generate name using AI and submit it

In this scenario let's consider that we already have access to a related OpenAPI Specification for a third party system that has the data we would like to import in our software. This approach is for demonstration purposes as the user should not be obligated to look up api specifications for the data he is looking for but just provide related keywords and choose among alternatives. To solve this problem in future versions, <https://apis.guru/>, a repository of publicly accessible OpenAPI Specifications, will be properly integrated to facilitate the search for the ideal third-party.



The screenshot shows a dark-themed web interface for 'PowerPulse'. The main heading is 'PowerPulse' in a large, white, sans-serif font. Below it, the sub-heading is 'Submit an OpenAPI Specification URL(json)'. There is a text input field containing the URL 'https://api.ember-climate.org/openapi.json'. Below the input field is a dark button with the text 'Submit' in white. Underneath the button, there is a line of text that reads 'Url currently is: some weird page' and another line that reads 'Not Yet Validated'.

Figure 7.4: Step 4: Submit an OpenAPI Specification to import the data

Now we have access to the API endpoints and we can choose to include only those necessary for our use case.

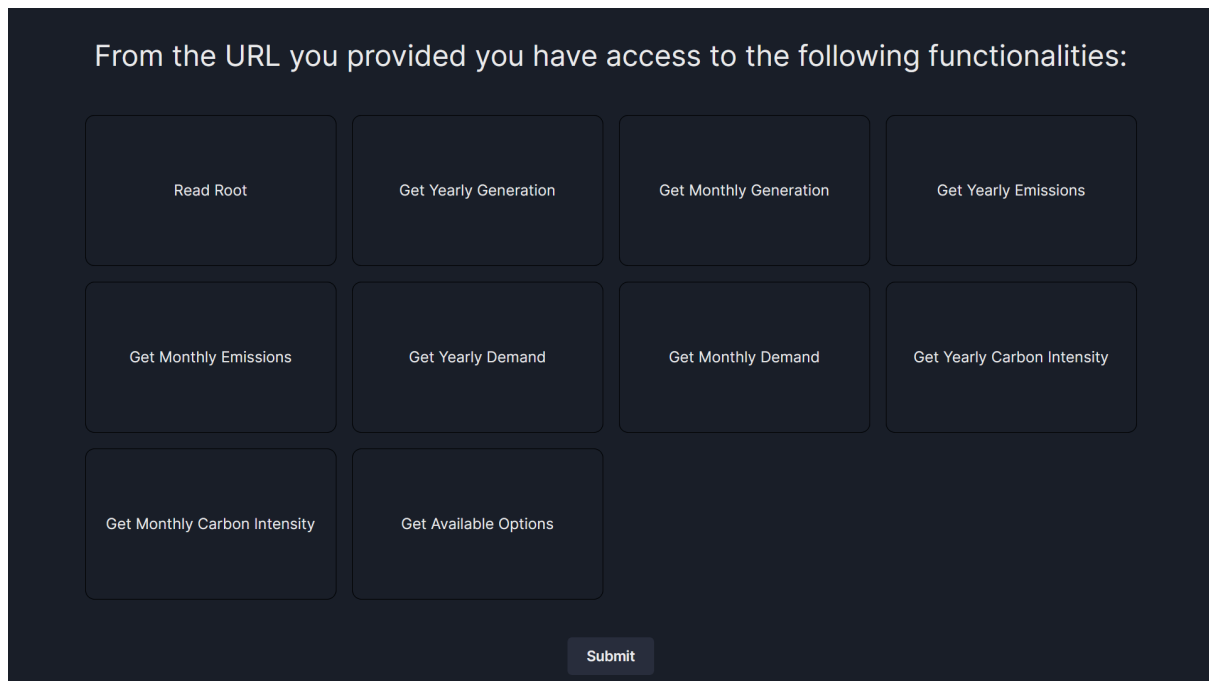


Figure 7.5: Step 5: See specific data types from data source

As we are trying to make financial trades on the energy markets we are mostly concerned with the amount of energy being generated and consumed by citizens to estimate how those numbers have evolved in the past and will evolve in the future.

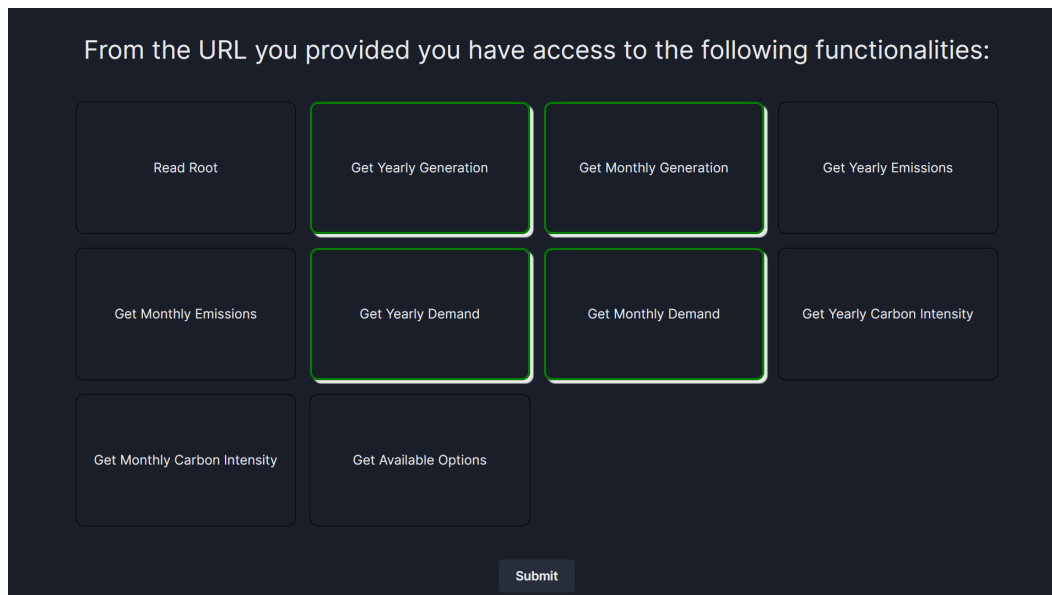


Figure 7.6: Step 6: Select data types

Here we can set metadata for the sources we have chosen above. In our situation the frequency and the type of data is obvious, but this is not commonly the case. Also AI can pre-fill this data for us for a smoother experience. We should also provide a time range for the data we want to keep in our database for analyses, as it is important for both statistical and storage cost purposes.

Figure 7.7: Step 7: Submit metadata for each data type

Now we can use the layout editor to create pages and add charts to each page. We are choosing mostly LineGraphs as we want to understand the time-series and detect trends. We create two pages, one for the generation data and one for the energy demand data. We are free to also combine those two in one page in a 2X2 formation according to our preferences.

Towards a domain-specific automated software development process

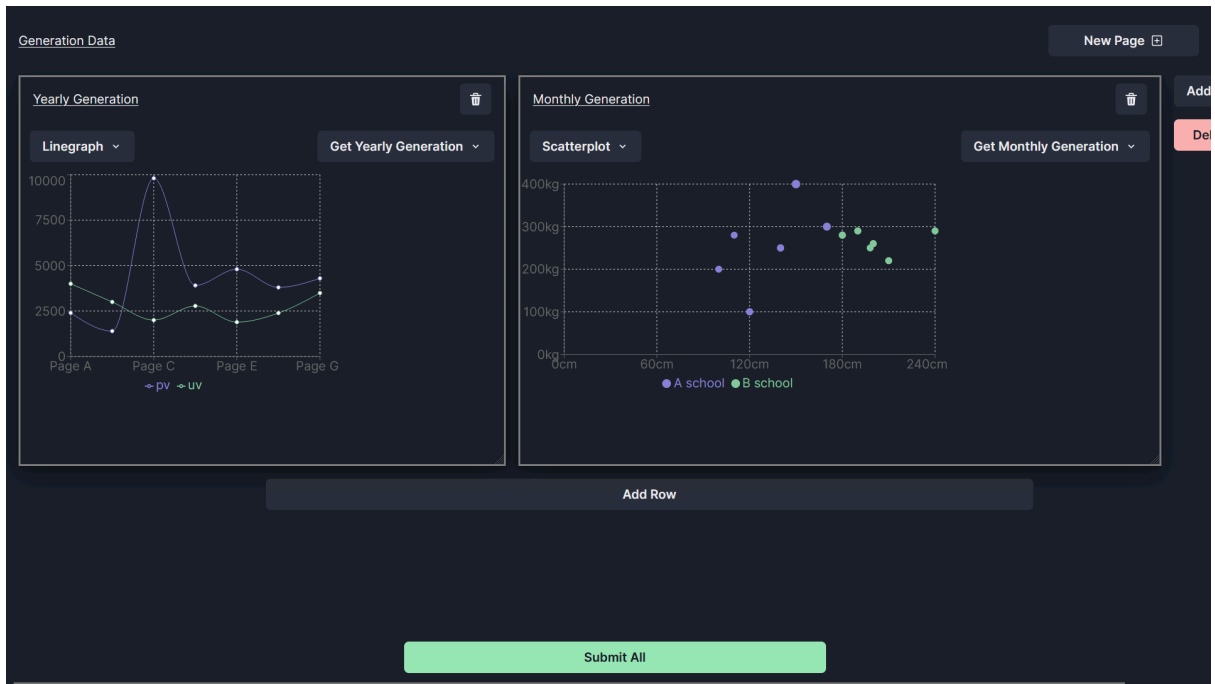


Figure 7.8: Step 8.1: Create layout for energy generation data

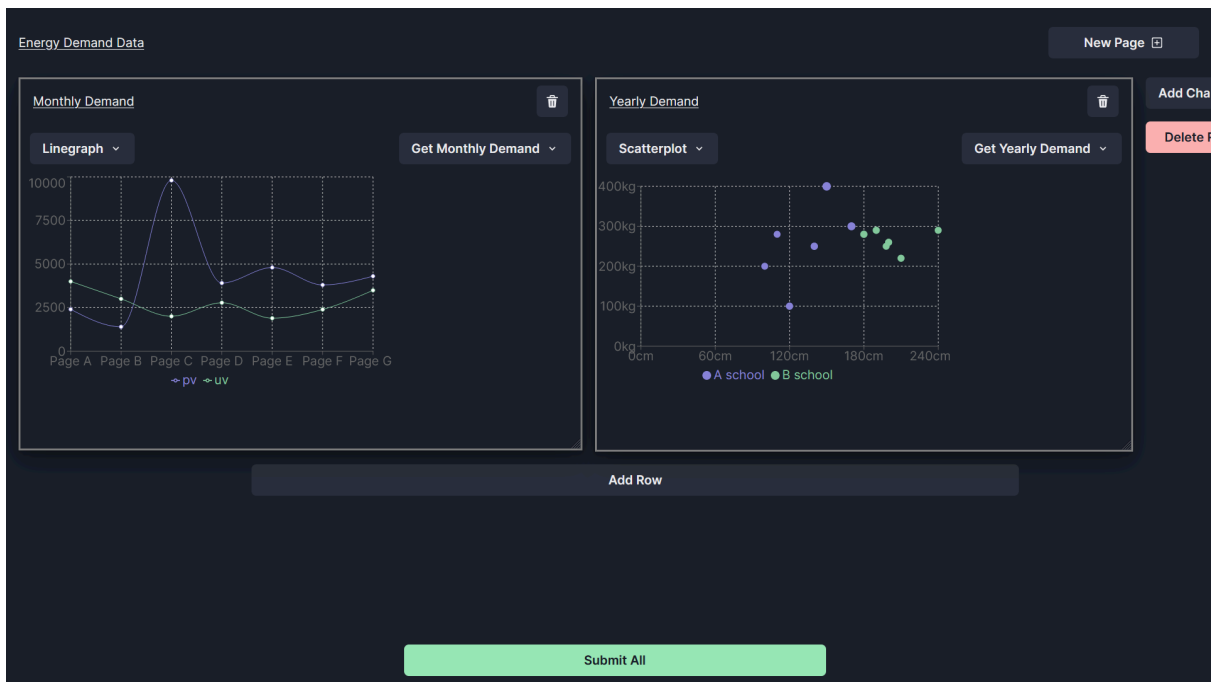


Figure 7.9: Step 8.2: Create layout for energy demand data

As it is important for the development process to provide upfront clear requirements in relation to the desired qualities of the software we can now fill in how much we care about different software qualities. In our case, mostly security and performance is what we are looking for so we fill in the data as demonstrated below.

The screenshot shows a web interface titled "Non Functional Requirements Page" with a dark background. It contains four sliders for different software qualities, each with a color-coded bar and a text description of the selected value.

Quality	Value	Description
Performance	60%	I care but let's not consume too much time for it
Maintainability	40%	I don't care much but just a bit never hurt anyone
Reliability	20%	For my scenario it barely matters
Security	100%	It is critical and it must be prioritised

A "Submit" button is located at the bottom center of the form.

Figure 7.10: Step 9: Submit Non-Functional Requirements for Big Data System

Now we are presented with two options, either generate a simple requirements document we can provide our developers to start the development process or generate the boilerplate(the repetitive parts of it) code to save developer man-hours.

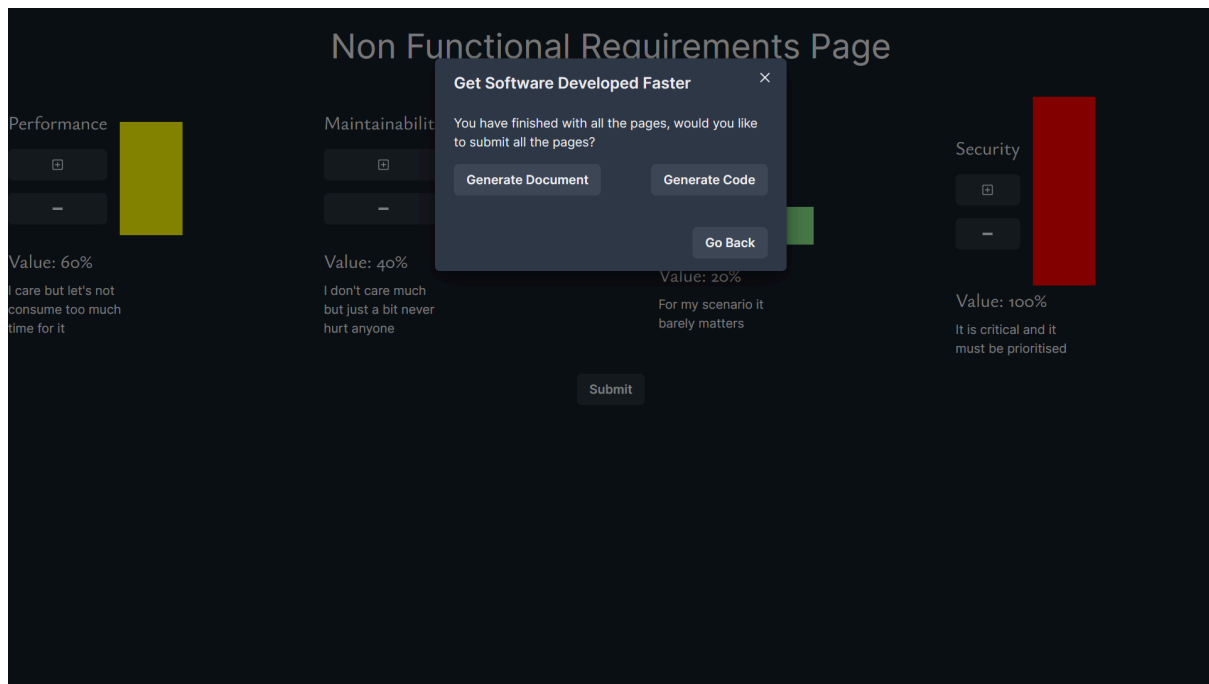


Figure 7.11: Step 10: Choose code generation or requirements document print.

By clicking on “Generate Document” we are receiving this document that informs our developers in a concise and unambiguous way.

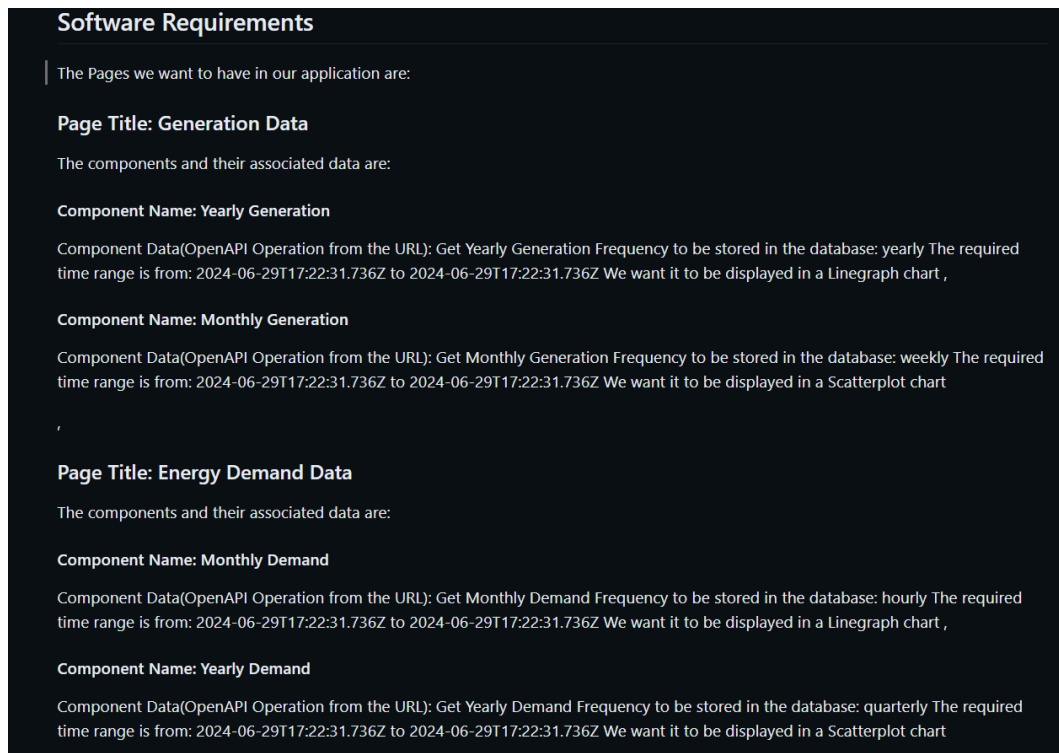


Figure 7.12: Step 11: Generated Requirements Document

By clicking “Generate Code” above we can now choose the programming language we intend to write this software in. Java is a popular programming language for enterprise applications so we will go with that. This step should be determined by the developers and it is only presented here in order to really understand the plethora of options available. (In future versions developers will provide a configuration of their preference for technical tools)

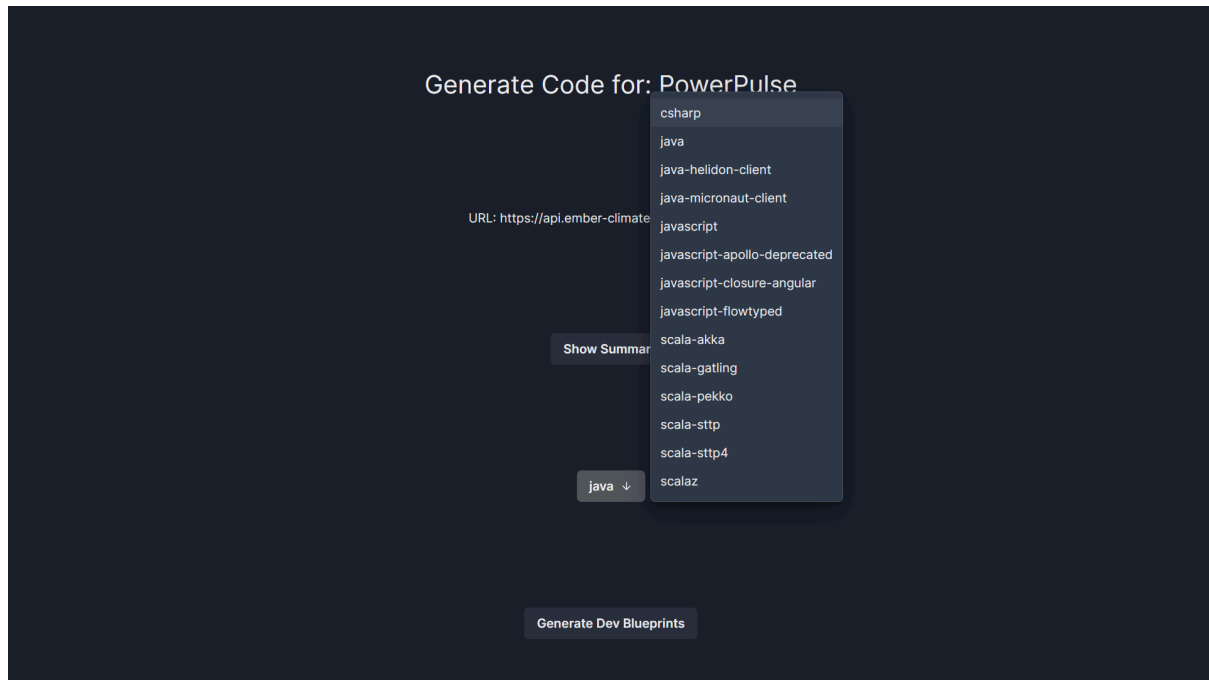


Figure 7.13: Step 12: Choose programming language for codegen

We can now see the created github repository, where the future code will reside.

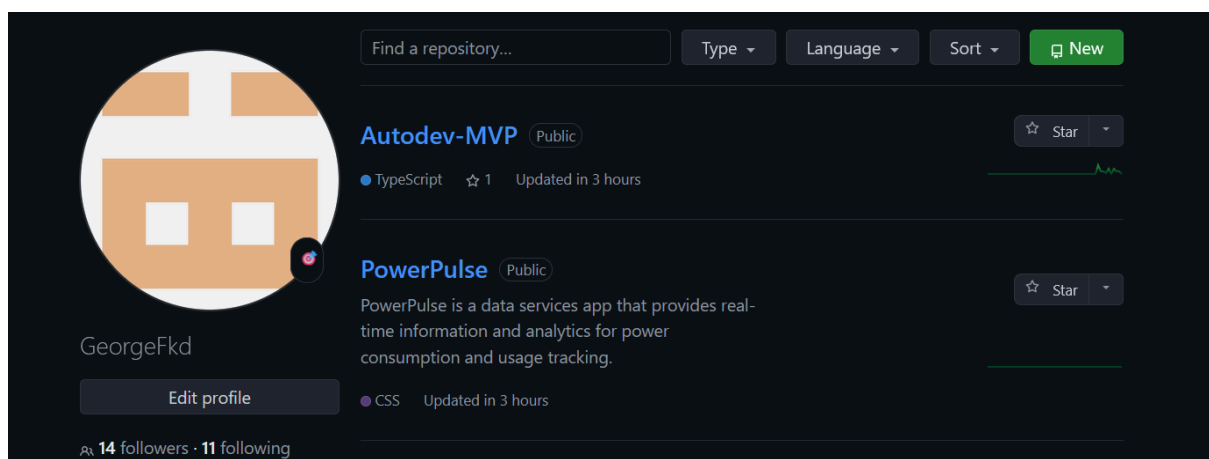


Figure 7.14: Github Repository created

Towards a domain-specific automated software development process

From the information provided in previous steps the development of the software has been analysed in separate tasks for developers to complete. In this situation one per page we created.

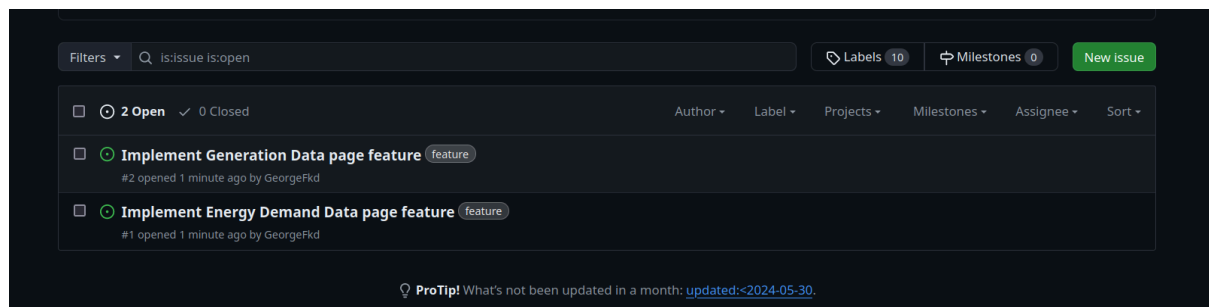


Figure 7.15: Github Issues generated from user input

Below we can see what is included as an analytical description of the task.

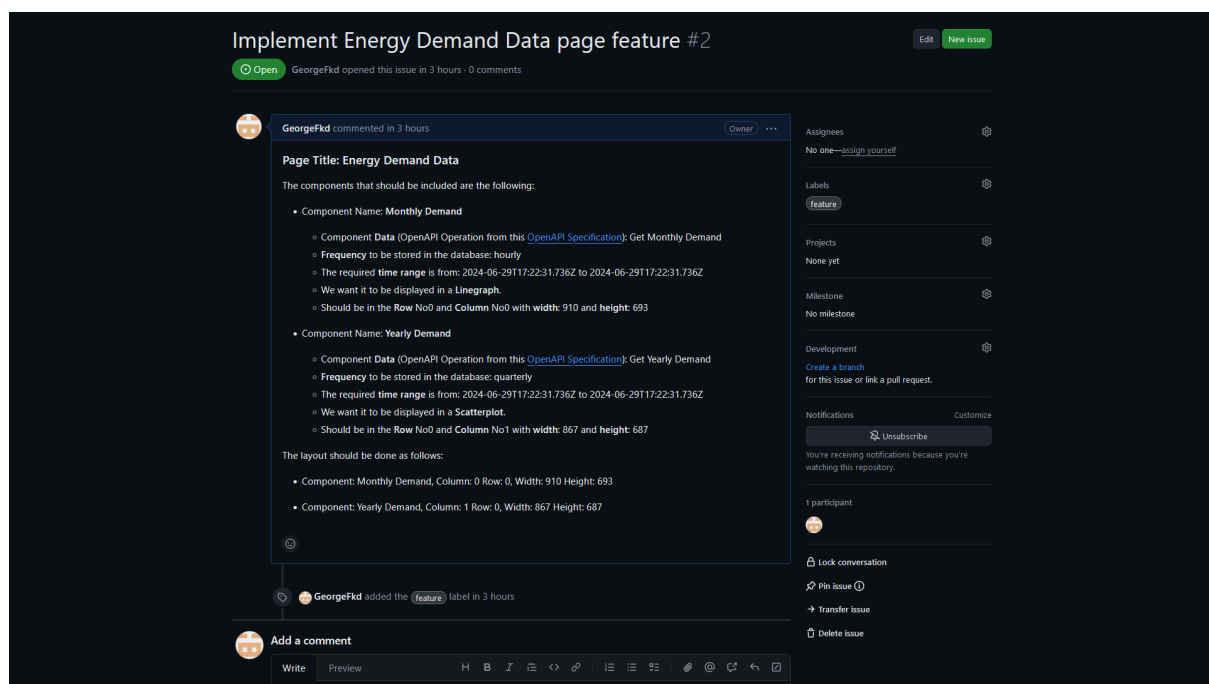


Figure 7.16: Details of Github Issue #1

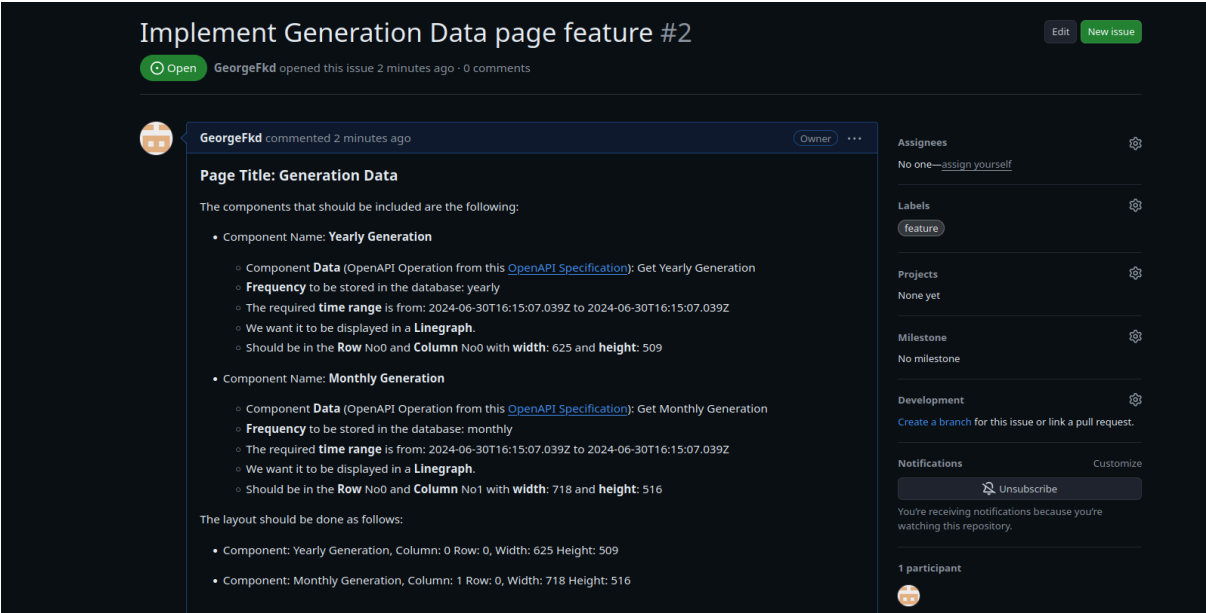


Figure 7.17: Details of Github Issue #2

Here we can see the frontend code(UI code) that was automatically generated. It also includes an API folder where the communication with the backend is implemented in order to save developer time. Also inside the pages folder there is boilerplate code for the layouts we defined previously.



Figure 7.18: Frontend code generated in github repository based on user input

Towards a domain-specific automated software development process

Here we can see the backend code for our system where the third-party system is already imported and the developers can use it in their custom code without extra effort.



	Name	Last commit message	Last commit da...
backend/third-party/EmberEnergy...	..		
> .github/workflows	.github/workflows	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
> .openapi-generator	.openapi-generator	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
> api	api	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
> docs	docs	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
> gradle/wrapper	gradle/wrapper	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
> main	src	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
> test/java/org/openapitools/clie...	src	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
.gitignore	.gitignore	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
.openapi-generator-ignore	.openapi-generator-ignore	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
.travis.yml	.travis.yml	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
README.md	README.md	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
build.gradle	build.gradle	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
build.sbt	build.sbt	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
git_push.sh	git_push.sh	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
gradle.properties	gradle.properties	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
gradlew	gradlew	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
gradlew.bat	gradlew.bat	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
pom.xml	pom.xml	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago
settings.gradle	settings.gradle	Initializing Github Repository with DevAutom and OpenAPI online	3 weeks ago

Figure 7.19: Backend code generated in github repository based on user input

Chapter 8: Discussion & Conclusions

8.1 Limitations

The major limitation of this work is the lack of an industry-based assessment of the tool in its effectiveness and cost/time saving benefits. In Chapter 4.2 the business context could be validated by industry research and feedback in order to expand it and understand it more deeply. In addition, the tool created is a proof-of-concept and thus has many areas of functional improvement in order to accommodate more complex Big Data Systems which will be added in future versions. Lastly, this work did not integrate existing model driven approaches in a theoretical and a practical sense, which will be addressed in future work of the author.

8.2 Further Research

As can be seen by the limitations of this work, further research includes the integration of this framework into existing programming ecosystems and tooling. Additionally, it should be validated quantitatively in order to bring to the surface its shortcomings in industrial software development settings. Finally, more industry research should be pursued to deepen the understanding of the requirements of Big Data Systems and expand the specification presented here.

References

- [1] “Most Common Software Development Challenges & How to Solve Them.” Accessed: Jul. 06, 2024. [Online]. Available: <https://www.orientsoftware.com/blog/software-development-challenges/>
- [2] S. Schulz, “47 Software Development Statistics, Trends, & Insights [Updated for 2024],” Don’t Do It Yourself. Accessed: Jul. 06, 2024. [Online]. Available: <https://ddiy.co/software-development-statistics/>
- [3] “Systems Analysis and Design: An Object-Oriented Approach with UML, 6th Edition | Wiley,” Wiley.com. Accessed: Jul. 06, 2024. [Online]. Available: <https://www.wiley.com/en-us/Systems+Analysis+and+Design%3A+An+Object-Oriented+Approach+with+UML%2C+6th+Edition-p-9781119559917>
- [4] M. A. Awad, “A comparison between agile and traditional software development methodologies,” *Univ. West. Aust.*, vol. 30, pp. 1–69, 2005.
- [5] W. W. Royce, “Managing the development of large software systems: concepts and techniques,” in *Proceedings of the 9th international conference on Software Engineering*, 1987, pp. 328–338. Accessed: Nov. 14, 2023. [Online]. Available: <https://blog.jbrains.ca/assets/articles/royce1970.pdf>
- [6] P. Kruchten, *The rational unified process: an introduction*. Addison-Wesley Professional, 2004. Accessed: Nov. 14, 2023. [Online]. Available: <https://books.google.com/books?hl=el&lr=&id=RYCMx6o47pMC&oi=fnd&pg=PR13&dq=unified+process+software+development+methodology&ots=h4ajwWSb-l&sig=n4PWq84227UtZRAMzBHzearpRI0>
- [7] B. W. Boehm, “A Spiral Model of Software Development and Enhancement,” 1988.
- [8] K. Schwaber, “SCRUM Development Process,” in *Business Object Design and Implementation*, J. Sutherland, C. Casanave, J. Miller, P. Patel, and G. Hollowell, Eds., London: Springer London, 1997, pp. 117–134. doi: 10.1007/978-1-4471-0947-1_11.
- [9] K. Beck, *Extreme programming explained: embrace change*. addison-wesley professional, 2000. Accessed: Nov. 14, 2023. [Online]. Available:

<https://books.google.com/books?hl=el&lr=&id=G8EL4H4vf7UC&oi=fnd&pg=PR13&dq=Extreme+Programming+Explained:+Embrace+Change&ots=jcsDrufXyo&sig=G9teMDW5O6xPyyFscqCrhp4hS8l>

- [10] F. Horozal, P. Reimer, and S. Scholze, "Tool Support for Architectural Pattern Selection and Application in Cloud-Centric Service-Oriented IDEs," in *Proceedings of the 3rd Eclipse Security, AI, Architecture and Modelling Conference on Cloud to Edge Continuum*, Ludwigsburg Germany: ACM, Oct. 2023, pp. 53–61. doi: 10.1145/3624486.3624494.
- [11] S. Farshidi, S. Jansen, and J. M. Van Der Werf, "Capturing software architecture knowledge for pattern-driven design," *J. Syst. Softw.*, vol. 169, p. 110714, Nov. 2020, doi: 10.1016/j.jss.2020.110714.
- [12] A. Ghazarian, "Characterization of functional software requirements space: The law of requirements taxonomic growth," in *2012 20th IEEE International Requirements Engineering Conference (RE)*, Sep. 2012, pp. 241–250. doi: 10.1109/RE.2012.6345810.
- [13] S. Farshidi and S. Jansen, "A Decision Support System for Pattern-Driven Software Architecture," in *Software Architecture*, vol. 1269, H. Muccini, P. Avgeriou, B. Buhnova, J. Camara, M. Caporuscio, M. Franzago, A. Koziolk, P. Scandurra, C. Trubiani, D. Weyns, and U. Zdun, Eds., in *Communications in Computer and Information Science*, vol. 1269. , Cham: Springer International Publishing, 2020, pp. 68–81. doi: 10.1007/978-3-030-59155-7_6.
- [14] S. Farshidi, S. Jansen, R. De Jong, and S. Brinkkemper, "A decision support system for software technology selection," *J. Decis. Syst.*, vol. 27, no. sup1, pp. 98–110, May 2018, doi: 10.1080/12460125.2018.1464821.
- [15] S. Farshidi, S. Jansen, and M. Deldar, "A decision model for programming language ecosystem selection: Seven industry case studies," *Inf. Softw. Technol.*, vol. 139, p. 106640, 2021.
- [16] C. Ouyang, M. Dumas, W. M. van der Aalst, A. H. Ter Hofstede, and J. Mendling, "From business process models to process-oriented software systems: The BPMN to BPEL way," *BPM Cent. Rep. BPM-06-27 BPMcenter Org*, 2006, Accessed: Nov. 03, 2023. [Online]. Available:

https://www.researchgate.net/profile/Marlon-Dumas/publication/27466650_From_Business_Process_Models_to_Process-oriented_Software_Systems_The_BPMN_to_BPEL_Way/links/02bfe50f3e2ab469df000000/From-Business-Process-Models-to-Process-oriented-Software-Systems-The-BPMN-to-BPEL-Way.pdf

- [17] J. M. Barnes, A. Pandey, and D. Garlan, “Automated planning for software architecture evolution,” in *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Nov. 2013, pp. 213–223. doi: 10.1109/ASE.2013.6693081.
- [18] C. B. Burlò, A. Francalanza, A. Scalas, and E. Tuosto, “COTS: Connected OpenAPI Test Synthesis for RESTful Applications,” in *Coordination Models and Languages*, vol. 14676, I. Castellani and F. Tiezzi, Eds., in Lecture Notes in Computer Science, vol. 14676. , Cham: Springer Nature Switzerland, 2024, pp. 75–92. doi: 10.1007/978-3-031-62697-5_5.
- [19] H. Ed-Douibi, J. L. C. Izquierdo, and J. Cabot, “Automatic generation of test cases for REST APIs: A specification-based approach,” in *2018 IEEE 22nd international enterprise distributed object computing conference (EDOC)*, IEEE, 2018, pp. 181–190. Accessed: Jun. 29, 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8536162/>
- [20] S. Karlsson, A. Čaušević, and D. Sundmark, “QuickREST: Property-based test generation of OpenAPI-described RESTful APIs,” in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*, IEEE, 2020, pp. 131–141. Accessed: Jun. 29, 2024. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9159071/>
- [21] D. Wolters, J. Kirchhoff, and G. Engels, “Specifying Web Interfaces for Command-Line Applications Based on OpenAPI,” in *Service-Oriented Computing – ICSOC 2019 Workshops*, vol. 12019, S. Yangui, A. Bouguettaya, X. Xue, N. Faci, W. Gaaloul, Q. Yu, Z. Zhou, N. Hernandez, and E. Y. Nakagawa, Eds., in Lecture Notes in Computer Science, vol. 12019. , Cham: Springer International Publishing, 2020, pp. 30–41. doi: 10.1007/978-3-030-45989-5_3.
- [22] B. A. Sanchez, K. Barmpis, P. Neubauer, R. F. Paige, and D. S. Kolovos, “Restmule: enabling resilient clients for remote APIs,” in *Proceedings of the 15th International Conference on Mining Software Repositories*, Gothenburg Sweden:

- ACM, May 2018, pp. 537–541. doi: 10.1145/3196398.3196405.
- [23] S. Barakat, E. Barba Roque, A. B. Sánchez Jerez, and S. Segura Rueda, “Automated code generation for inter-parameter dependencies in REST APIs,” *XVII Jorn. Ing. Cienc. E Ing. Serv. JCIS 2022* 2022 Pp 1-4, 2022, Accessed: Jun. 29, 2024. [Online]. Available: <https://idus.us.es/handle/11441/153727>
- [24] A. Martin-Lopez, “Automated analysis of inter-parameter dependencies in web APIs,” in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*, Seoul South Korea: ACM, Jun. 2020, pp. 140–142. doi: 10.1145/3377812.3382173.
- [25] A. Aldazabal, T. Baily, F. Nanclares, A. Sadovykh, C. Hein, and T. Ritter, “Automated model driven development processes,” in *Proceedings of the ECMDA workshop on Model Driven Tool and Process Integration*, Citeseer, 2008, pp. 361–375. Accessed: Jun. 30, 2024. [Online]. Available: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=fb44e76891ab47c7e73b37ebd45ef8f6fa776893>
- [26] “Meet Figma AI: Empowering Designers with Intelligent Tools | Figma Blog,” Figma. Accessed: Jul. 06, 2024. [Online]. Available: <https://www.figma.com/blog/introducing-figma-ai/>
- [27] D. Mairiza, D. Zowghi, and N. Nurmuliani, “An investigation into the notion of non-functional requirements,” in *Proceedings of the 2010 ACM Symposium on Applied Computing*, Sierre Switzerland: ACM, Mar. 2010, pp. 311–317. doi: 10.1145/1774088.1774153.
- [28] L. Chung and J. C. S. Do Prado Leite, “On Non-Functional Requirements in Software Engineering,” in *Conceptual Modeling: Foundations and Applications*, vol. 5600, A. T. Borgida, V. K. Chaudhri, P. Giorgini, and E. S. Yu, Eds., in Lecture Notes in Computer Science, vol. 5600. , Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 363–379. doi: 10.1007/978-3-642-02463-4_19.
- [29] D. Gross and E. Yu, “From Non-Functional Requirements to Design through Patterns,” *Requir. Eng.*, vol. 6, no. 1, pp. 18–36, Feb. 2001, doi: 10.1007/s007660170013.